

Codity — Distributed Job Scheduler

Intern Technical Assignment Submission

Author: Surianandhan **Date:** 2026-08-20 **Repository:** <https://github.com/Surianandhan/Codity>

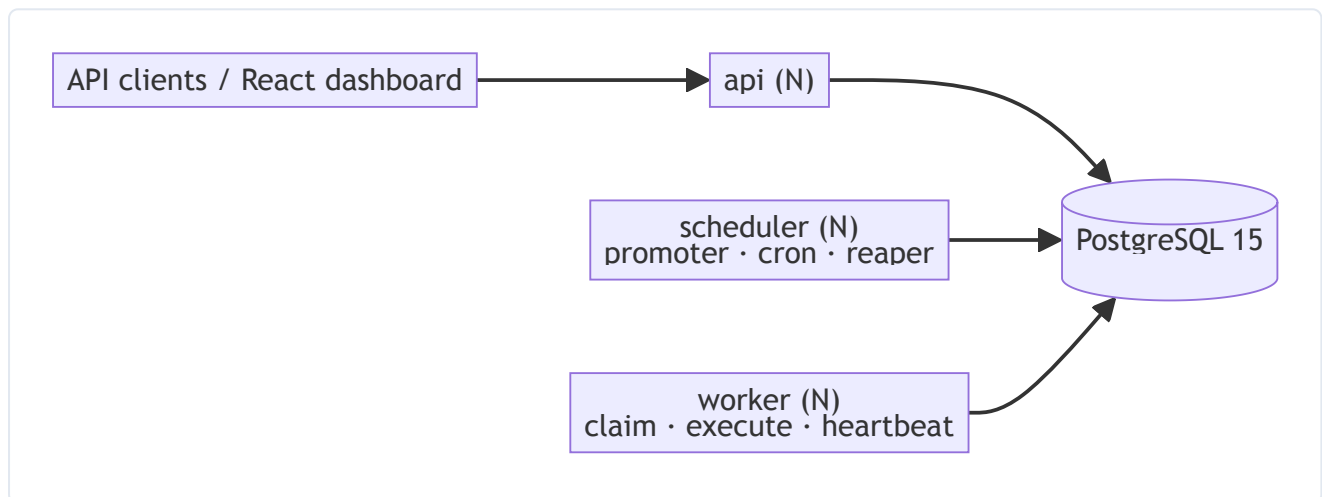
A production-inspired distributed job scheduling platform on PostgreSQL 15 — no Redis, no broker, no Docker. Jobs are claimed with `SELECT ... FOR UPDATE SKIP LOCKED ;` reliability is enforced with database constraints, not application discipline alone.

This document combines the project's own documentation — written and adversarially reviewed before a line of code was written — with evidence from running the live system, including one genuine `kill -9` recovery. It is a complete written submission; the linked repository holds the executable source.

Overview & Setup

A production-inspired background job platform: submit jobs over a REST API, and a fleet of worker processes claims and executes them across machines. Jobs can be immediate, delayed, scheduled, recurring (cron), or submitted in batches. Failures retry with jittered backoff, permanent failures land in a dead letter queue an operator can inspect and replay, and a worker that dies mid-job has its work reclaimed and finished by another — **without ever running the job twice against the same attempt**.

The architecture in three sentences. Three stateless process types — `api`, `worker`, `scheduler` — share nothing but a single PostgreSQL 15 database; there is no Redis, no Celery, and no broker. Workers claim jobs with a single `SELECT ... FOR UPDATE SKIP LOCKED` statement that transitions the job and opens its execution row atomically, so K workers partition the ready set with no coordination and no blocking. Every ownership change bumps a `lease_epoch` fencing token, so a worker that stalls and is reclaimed can never commit a stale result — which is what makes crash recovery safe rather than merely likely.



⚠ Nothing executes without a running worker

This is the single most likely way to conclude the system is broken when it is working exactly as designed.

The API **accepts** jobs and returns `201`. It never executes them. A job stays in `status: "queued"` until a `worker` process claims it. If you POST a job with no worker running, you will see a job that never moves — that is correct behaviour, not a bug.

You need **three** processes, in three terminals: `make api`, `make worker`, `make scheduler`.

The scheduler is not optional either. Without it, *immediate* jobs still flow perfectly while every *delayed* job, every *cron* occurrence, and **every backoff retry** stalls silently. That is the most confusing failure the system can have, which is why the scheduler's last-tick age is reported at `GET /api/v1/system/status` and shown on the dashboard.

Quickstart — no Docker required

macOS with Homebrew, PostgreSQL 15, Python 3.13, and `uv`. Docker is not used anywhere in this project, including in the tests.

Start PostgreSQL:

```
brew services start postgresql@15
```

Create the two databases (the second is for the test suite):

```
createdb codity && createdb codity_test
```

Install Python dependencies:

```
cd backend && uv sync
```

Apply migrations:

```
cd backend && uv run alembic upgrade head
```

Seed a demo organization, project, queues and handlers:

```
cd backend && uv run python scripts/seed.py
```

The seed prints an **organization id** and a login. Keep the org id — the worker needs it, because a worker connects to Postgres directly and must be told which tenant it serves.

Run the system — four terminals

API:

```
cd backend && uv run uvicorn app.main:app --reload --port 8000
```

Worker (substitute the org id printed by the seed):

```
cd backend && uv run python -m app.worker.main --org "$CODITY_ORG_ID" --name worker-1 --concurrency 1
```

Scheduler:

```
cd backend && uv run python -m app.scheduler.main
```

Dashboard:

```
cd frontend && npm install && npm run dev
```

Then open <http://localhost:5173> for the dashboard and <http://localhost:8000/docs> for Swagger.

Or use the Makefile

Every command above has a target. `make` on its own lists them.

```
make setup && make migrate && make seed
```

```
make api · make worker · make worker2 · make scheduler · make demo · make test · make check
```

`make worker` resolves the org id from the local database automatically; override with `make worker ORG=<uuid>`.

See it work

[docs/GRADER_WALKTHROUGH.md](#) is a **numbered ~10 minute path** through every capability, with each step annotated by what it demonstrates. The short version:

```
make demo
```

500 jobs across 3 queues with a handler that fails ~20% of the time. The throughput chart moves, failures retry with visibly increasing gaps, and some jobs reach the DLQ. At the end it prints an invariant block:

```
duplicate first-attempt executions = 0  
terminal == enqueued
```

Then start a second worker, re-run the load, and send `SIGKILL` to the first one mid-flight. Within one lease period the reaper reclaims its jobs, worker-2 finishes them, and the job timeline shows

```
attempt 1 · claimed by worker-1 → lease expired (lost)  
attempt 2 · claimed by worker-2 → running → completed
```

with the invariant block still clean. That is the whole design in one screen.

Documentation

Document	What is in it
docs/ARCHITECTURE.md	Process topology, architecture diagram, module layout, the layering rule, observability
docs/DATABASE.md	ER diagram, table-by-table rationale, every index and the query it serves, the status state machine
docs/DESIGN_DECISIONS.md	ADR-lite: 17 decisions, each with the option rejected and why
docs/API.md	Endpoints, auth, error envelope, keyset pagination, idempotency
docs/SCHEMA_NAMES.md	The canonical data dictionary. Every other document cites it
docs/TESTING.md	What is tested, why the concurrency fixtures are separate, and what is deliberately not tested
docs/GRADER_WALKTHROUGH.md	The 10-minute tour

Diagrams are inline Mermaid, so they render on GitHub and change in the same diff as the code they describe.

Reliability, in one table

Failure	Detection	Recovery	Residual risk
Worker killed mid-job	<code>lease_expires_at < now()</code>	Reaper closes the execution as <code>lost</code> , requeues the job, bumps the epoch	Side effects from the dead attempt already happened — handler idempotency covers it
Worker slow, stopped, or partitioned	Heartbeat stops	Reaper reclaims; the zombie's writes are rejected by the epoch fence	The zombie's external side effect cannot be un-sent
Database connection lost mid-job	asyncpg raises	Job never transitions; the lease expires; the reaper requeues	One extra attempt consumed only if the job had started
Job exceeds <code>timeout_ms</code>	<code>asyncio.wait_for</code>	Marked <code>timed_out</code> , counts as a failed attempt, backs off	A blocking handler runs to completion in its thread; bounded by <code>timeout_ms < lease_seconds × 1000</code>
Poison-pill job	Attempt budget exhausts	Dead-lettered with an error fingerprint; the DLQ groups by fingerprint	None
Two schedulers race a cron occurrence	—	<code>UNIQUE (schedule_id, scheduled_for)</code>	None
Clock skew between nodes	—	All time comes from <code>now()</code> inside Postgres; no process compares its own clock to a lease	None by construction

Delivery is at-least-once, and the docs say so. Exactly-once across a process boundary is impossible — a worker cannot atomically commit "job done" to Postgres and "email sent" to a third-party API. What *is* guaranteed: at most one worker holds a valid lease at any instant, at most one execution can record a result for a given `lease_epoch`, and handlers are given a written idempotency contract. See [ADR-005](#).

Deliberately deferred

Each of these was designed, costed, and cut on purpose. The design and the migration path for each is written down, so none of them is a surprise later.

Deferred	Why	Where the design lives
PostgreSQL RLS	A <i>second</i> enforcement layer for something already enforced by composite foreign keys and an automatic query predicate, and asserted by a CI test. Roughly a day, and every fixture or migration that forgets to set the GUC becomes an opaque "zero rows, no error" debugging session. That day went to the reliability core instead.	ADR-013 — including the exact policy DDL
<code>api_keys</code> (service-to-service auth)	Nothing consumes it yet. The dashboard uses JWT, and workers connect to Postgres directly rather than through the API, so an API key would be an unused table with an unused rotation story. It becomes necessary the moment a third-party service posts jobs.	docs/API.md §2
Four-tier RBAC (owner/admin/operator/viewer)	RBAC is listed as a <i>bonus</i> . A four-tier ladder puts a bonus on the critical path of every endpoint and multiplies the auth test matrix by four. Two roles (owner, member) ship; the ladder slots into the same dependency as <code>require_role(min_role)</code> with no schema change beyond widening a CHECK.	ADR-014
WebSocket live updates	Also a listed <i>bonus</i> . Reconnect-with-backoff, auth on upgrade, resubscribe, server-side fanout, and missed-event reconciliation are a lot of subtle code. Polling is correct by construction — every poll re-reads the truth — and its cost is bounded: terminal jobs stop polling, hidden tabs pause. The polling hooks are the seam if the transport is swapped later.	ADR-015
Table partitioning	Monthly declarative partitioning of <code>job_logs</code> and <code>job_executions</code> turns retention into a partition detach. At this volume the batched sweep is measurably sufficient, and partitioning would add migration complexity with no observable benefit.	docs/DATABASE.md §7
Queue sharding	Claims are serialised <i>per queue</i> by the <code>FOR NO KEY UPDATE</code> lock that makes <code>max_concurrency</code> exact. Hashing <code>queue_id</code> into shards removes that serialisation — worth doing when one queue needs more claim throughput than one short statement provides, and not before.	ADR-003
Duration percentiles	The per-minute rollup stores <code>sum_duration_ms</code> and <code>max_duration_ms</code> , so mean and max are	docs/ARCHITECTURE.md §5

Deferred	Why	Where the design lives
	available. True percentiles need a second rollup pass over raw durations.	

An exactly-once *delivery* guarantee is not on this list, because it is not deferred — it is impossible, and claiming it would be the least honest thing in the repository.

Development

```
make check
```

ruff + mypy (strict) + the full test suite. Run it before every commit.

```
make test-concurrency
```

Just the concurrency tests, which run against real committed sessions — one connection per simulated worker, because `SKIP LOCKED` semantics do not exist inside a single transaction.

Tests run against a real local `codity_test` database. **No testcontainers**, because Docker is not available here and because the behaviour under test is genuine PostgreSQL locking, which does not survive being mocked. Details in [docs/TESTING.md](#).

Troubleshooting

A job stays `queued` and never runs. No worker is running, or the worker is subscribed to a different queue or a different organization. Check the worker terminal for claim logs, and check `GET /api/v1/orgs/{org_id}/workers` — a queue with no assigned worker is indistinguishable from an idle queue unless you look.

Delayed, scheduled, or retried jobs never become `queued`, but immediate jobs work fine. The scheduler is not running. Confirm with `GET /api/v1/system/status`: `promoter.last_run_at` will be stale. Start it with `make scheduler`.

`createdb: database "codity" already exists` Harmless — it already exists. `make setup` tolerates this.

`connection refused on port 5432`. PostgreSQL is not running: `brew services start postgresql@15`. Confirm with `psql -l`.

`FATAL: database "codity_test" does not exist when running tests`. Run `createdb codity_test`, or `make setup`.

`alembic upgrade head` reports a conflicting head. Two migration branches exist. `uv run alembic heads` shows them; they need a merge revision.

Everything is slow and jobs claim in bursts. Expected under a low `max_concurrency` — the cap is exact, and claims for one queue are serialised by design. Raise `max_concurrency`, or spread the load across more queues ([ADR-003](#)).

A job says `dead_letter` but the handler looks correct. Check `last_error_class`. `LeaseExpired` means the lease ran out rather than the handler failing — usually a blocking handler that did not use `asyncio.to_thread` and froze the event loop, which starves the heartbeat for every job on that worker.

The dashboard shows 401 after a page reload. The access token is short-lived and refreshed from an `httpOnly` cookie. If the cookie is missing, the API and the dashboard are on origins the CORS config does not allow — check `CODITY_CORS_ORIGINS`.

A test that spawns concurrent workers passes suspiciously. It is probably using the rollback `db` fixture. Concurrency tests need `db_committed` and separate sessions; see [docs/TESTING.md](#) §2.

Stack

Python 3.13 · FastAPI · Pydantic v2 · SQLAlchemy 2.0 (async) · Alembic · asyncpg · PostgreSQL 15 · React + TypeScript + Vite + TanStack Query + Tailwind · `uv`

PostgreSQL is the only infrastructure dependency. That is the point.

Architecture

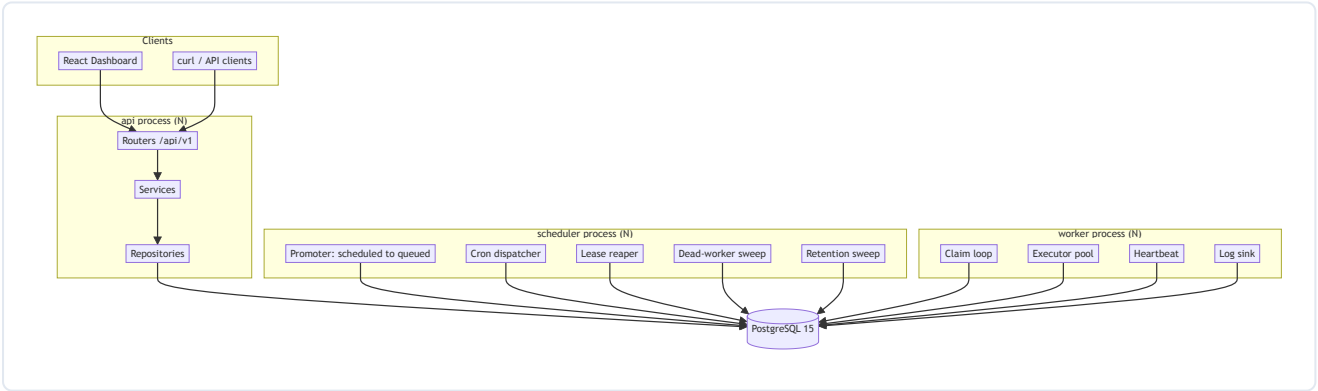
Codity is a distributed job scheduler whose only infrastructure dependency is PostgreSQL 15. Jobs are submitted over a REST API and executed by a fleet of worker processes that claim work with `SELECT ... FOR UPDATE SKIP LOCKED`. There is no Redis, no Celery, and no broker.

Names used below are defined once in `SCHEMA_NAMES.md`. Rationale for each choice is in `DESIGN_DECISIONS.md`.

1. Process topology

Three process types. All are stateless, all speak only to Postgres, and all can be scaled to N replicas without coordination.

Process	Owns	Scaling
api	FastAPI/uvicorn. HTTP, auth, validation, job creation, dashboard reads. Never claims jobs.	N replicas, stateless
worker	Claim → execute → complete/fail. Heartbeats. Graceful drain.	N replicas; this is the throughput knob
scheduler	Promoter, cron dispatcher, lease reaper, dead-worker sweep, retention sweep.	N replicas; correctness does not depend on N=1



Every arrow terminates at Postgres. The processes never talk to each other. That is the whole coordination story: **all shared state, all mutual exclusion, and all ordering live in the database**, which is why any of the three can be killed, restarted, or duplicated without a protocol to renegotiate.

Why the scheduler is its own process

It is not an API background task, because that ties job liveness to HTTP traffic — a system with no requests stops promoting delayed jobs — and it double-fires on every API replica.

It is not a leader-elected worker role, because leader election is a distributed-systems problem this system does not need to solve. Correctness under N schedulers comes from the database:

- **Cron double-promotion is impossible** — `UNIQUE (schedule_id, scheduled_for)` on `jobs`.
- **Reaper/promoter overlap is harmless** — every statement is `FOR UPDATE SKIP LOCKED`, so two schedulers *partition* the work instead of colliding.

A `pg_try_advisory_lock` is used only to stop redundant ticks burning CPU. If it is never acquired, the system is still correct. That is the test of whether a lock is load-bearing: remove it and see whether an invariant breaks. This one does not.

Worker identity and queue subscription

A worker connects to Postgres directly, so it must be told which tenant it serves: `--org <uuid>` (or `CODITY_ORG_ID`) is validated at startup. `--queues` takes `project-slug/queue-name` pairs, because queue names are unique per project and not globally.

On boot the worker upserts its `workers` row keyed on `(organization_id, name)` where the default name is `{hostname}-{pid}`, and populates `worker_queue_assignments`. A retention rule deletes `workers` rows in status `dead / stopped` older than 24h, so restarts do not leave the fleet screen full of corpses.

Per polling cycle the worker iterates its queues in **weighted-random order by** `queues.priority`, issuing one claim statement per queue and capping each queue's share at `ceil(batch_size / n_queues)`. Weighted-random rather than strict ordering is what stops a busy high-priority queue starving a low-priority one — and both properties are testable.

2. Module layout

```
Codity/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI app factory
│   │   ├── config.py             # pydantic-settings, CODITY_prefix
│   │   ├── domain/               # FastAPI-free: enums, errors, backoff, cron
│   │   ├── db/
│   │   │   ├── session.py
│   │   │   ├── models/           # SQLAlchemy models, one module per cluster
│   │   │   └── sql/              # raw .sql for the hot path
│   │   │       ├── claim_jobs.sql
│   │   │       ├── start_job.sql
│   │   │       ├── complete_job.sql
│   │   │       ├── fail_job.sql
│   │   │       ├── reap_leases.sql
│   │   │       ├── promote_due.sql
│   │   │       └── heartbeat.sql
│   │   ├── repositories/
│   │   ├── services/             # jobs, queues, claim, idempotency
│   │   ├── api/
│   │   │   ├── deps.py           # auth, scope resolution
│   │   │   ├── errors.py         # exception handlers, one envelope
│   │   │   ├── middleware/request_context.py
│   │   │   └── routers/          # auth, projects, queues, jobs, workers, dlq, metrics
│   │   ├── worker/
│   │   │   ├── main.py           # CLI entrypoint
│   │   │   ├── runner.py         # claim loop, executor pool, heartbeat, drain
│   │   │   ├── logsink.py        # the only writer of job_logs
│   │   │   └── handlers/         # echo, sleep, http_request, flaky, cpu_burn
│   │   └── scheduler/main.py     # promoter, cron, reaper, sweeps
│   ├── alembic/versions/
│   ├── scripts/                  # seed.py, demo_load.py, export_openapi.py
│   ├── tests/
│   └── pyproject.toml
├── frontend/
├── docs/
└── Makefile
```

The **hot path is raw SQL in .sql files**, not ORM expressions. The claim, start, complete, fail, reap, promote and heartbeat statements are each a single multi-CTE statement whose exact shape is the correctness argument. Keeping them as SQL means they are reviewable as SQL, can be pasted into `psql` and `EXPLAIN` ed directly, and cannot be silently restructured by a query-compiler change. The ORM owns CRUD, where its ergonomics pay off and its generated SQL does not matter.

3. The layering rule

```
routers → services → repositories → models
```

over a FastAPI-free `domain/`.

- `domain/` imports nothing from `app.api`, `app.db`, or `fastapi`. It is pure: enums, errors, backoff arithmetic, cron helpers.
- `services/` and `repositories/` never import `fastapi`. **This is what lets the worker and the scheduler reuse `services/claim.py` without dragging in the web framework** — the rule exists for that specific reason, not as decoration.
- Routers hold no business logic; they translate HTTP to service calls and back.
- Nothing outside `app/api/` raises `HTTPException`. Services raise `DomainError` subclasses, and `app/api/errors.py` is the only place that knows about status codes.

A test walks the AST of every module under `backend/app/` and asserts these import rules, so the layering is **enforced rather than aspirational**. A layering rule nobody checks is a comment.

4. Request and job lifecycle

1. `POST /api/v1/queues/{queue_id}/jobs` arrives. `RequestContextMiddleware` mints a `request_id` and binds it into structlog's contextvars.
2. The router validates the discriminated union on `kind`, resolves the principal, and calls `services/jobs.py`.
3. The service snapshots the queue's policy onto the job — `lease_seconds` from `queues.visibility_timeout_sec`, `timeout_ms`, `max_attempts`, backoff parameters, priority — and persists `request_id` as `jobs.correlation_id`. Snapshotting is what stops an edited queue or retry policy retroactively changing the backoff of jobs already mid-retry.
4. Status on creation: `immediate` and `batch` → `queued` with `run_at = now()`; `delayed`, `scheduled`, `recurring` → `scheduled`.
5. The promoter moves due `scheduled` rows to `queued`. A worker claims, starts, executes, and completes or fails. Failure with budget remaining goes back to `scheduled` with a jittered future `run_at`; exhausted budget dead-letters.

The mechanics of each transition — the claim statement, the epoch fence, the reaper, backoff — are in [DESIGN_DECISIONS.md](#) and [DATABASE.md](#).

5. Observability

Structured logs. `structlog` renders JSON. A `correlation_id` is generated per HTTP request, persisted on `jobs.correlation_id`, and re-bound by the worker when it claims — so **one `grep` follows a job from the POST that created it through every retry on every worker**, across process boundaries, without a tracing backend.

Health endpoints.

Endpoint	Answers
GET /healthz	Is the process alive? No I/O.
GET /readyz	Is the database reachable and are migrations current?
GET /api/v1/system/status	Worker count live/dead, oldest overdue scheduled job, DLQ depth, scheduler last-tick age .

Cross-process staleness. The last-tick age needs a home in the database, because the API and the scheduler are different processes — an in-process gauge is invisible to the endpoint that has to report it. A four-column table solves it:

```
CREATE TABLE system_state (
  name      text PRIMARY KEY,  -- 'promoter' | 'cron' | 'reaper' | 'retention'
  last_run_at timestampz NOT NULL,
  last_error text,
  updated_at timestampz NOT NULL DEFAULT now()
);
```

Every scheduler loop upserts its row on every tick. This converts the system's biggest silent-failure mode — **a dead promoter, which stalls every delayed job and every backoff retry while immediate jobs keep flowing perfectly** — from an invisible outage into a number on the dashboard. It is the one failure that looks like nothing is wrong.

Metrics. `queue_stats_minute` is a per-minute rollup (`enqueued` , `completed` , `failed` , `dead_lettered` , `sum_duration_ms` , `max_duration_ms`) written by the scheduler. The throughput chart reads the rollup, not `jobs` — a dashboard that aggregates the hot table is a dashboard that competes with the claim path for the same pages.

Database Design

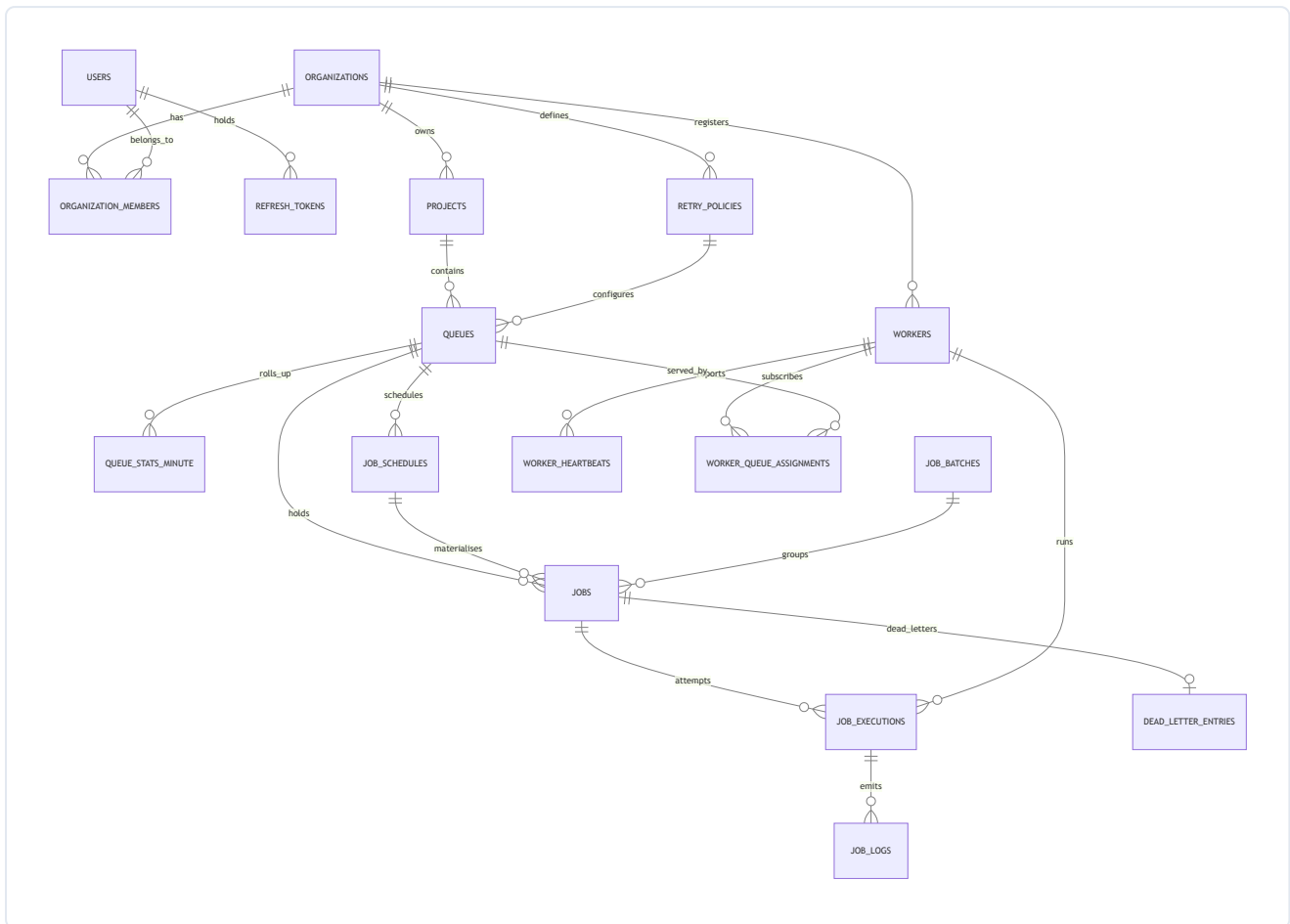
PostgreSQL 15 is the whole backend: system of record, queue substrate, lock manager, and clock. Column and value names come from [SCHEMA_NAMES.md](#) ; the rationale behind each structural choice is in [DESIGN_DECISIONS.md](#) .

Nineteen tables — eighteen entity tables plus the `queue_stats_minute` rollup. Tables land in migration order across build slices; `uv run alembic current` and `\dt in psql` show what is live in your database right now.

1. Conventions

- **UUIDv7, application-generated** for entity tables. Time-ordered, so inserts stay on the rightmost index page instead of dirtying a random leaf per row the way UUIDv4 does; non-enumerable, unlike a `bigint` ; and generable by the client before the round trip.
 - **`bigint GENERATED ALWAYS AS IDENTITY`** for the append-only high-volume children (`job_executions` , `job_logs` , `worker_heartbeats` , `queue_stats_minute`). They are never referenced externally, and 8 bytes beats 16 on the tables that grow fastest.
 - **All timestamps `timestampz`** . No naive timestamps anywhere, in the schema or in Python.
 - **Native `ENUM`** for `job_status` and `execution_status` , because they drive partial indexes and the set is closed. **`TEXT + CHECK`** everywhere else, so values can evolve without `ALTER TYPE` and its lock.
 - **`server_default=text(...)` , never a plain Python string**. A plain string is emitted as a DDL *literal* , which freezes the value at migration time — `now()` becomes the instant the migration ran. Every column a raw-SQL `INSERT` touches carries a server default, because raw SQL bypasses ORM defaults entirely.
 - **Composite foreign keys carry the tenant**. `jobs` has `UNIQUE (id, organization_id)` , and `job_executions` references `(job_id, organization_id)` . A cross-tenant child row is therefore physically impossible rather than merely unlikely.
-

2. Entity relationships



`idempotency_keys` and `system_state` are omitted from the diagram: neither has an entity relationship worth drawing. They are described in §3.

3. Table by table, and why it exists

Tenancy and identity

Table	Why
organizations	The tenant boundary. Every tenant-scoped table carries <code>organization_id</code> , and the composite FKs above make it non-forgeable.
users	Global identity. Argon2id password hash, plus <code>token_version</code> for global revocation on password change.
organization_members	Join table carrying <code>role</code> (<code>owner</code> <code>member</code>). PK (<code>organization_id</code> , <code>user_id</code>). A constraint trigger prevents removing or demoting the last owner — otherwise an org becomes permanently unadministrable through a normal UI action.
refresh_tokens	Rotating refresh tokens keyed by <code>jti</code> , with <code>replaced_by_jti</code> so reuse of a rotated token is detectable and can revoke the whole chain. Without the back-pointer, a stolen refresh token is indistinguishable from a legitimate one.
projects	Namespace inside an org. Queue names are unique per project, not globally — which is why the worker's <code>--queues</code> takes <code>project-slug/queue-name</code> .

Configuration

Table	Why
retry_policies	The spec names retry policies as an entity. Own table so a policy is reusable and nameable across queues — and its values are snapshotted onto <code>jobs</code> at enqueue , so editing a policy never retroactively changes the backoff of jobs already mid-retry. Referenced <code>ON DELETE RESTRICT</code> .
queues	The unit of concurrency, priority, lease length, payload limit, and pause. Carries <code>max_concurrency</code> , <code>priority</code> , <code>default_priority</code> , <code>visibility_timeout_sec</code> , <code>default_timeout_ms</code> , <code>max_payload_bytes</code> , <code>is_paused</code> / <code>paused_at</code> , <code>log_retention_days</code> .

Two CHECKs on `queues` earn their place:

```

CONSTRAINT ck_queues_pause CHECK (is_paused = (paused_at IS NOT NULL))
CONSTRAINT ck_queues_timeout_lt_lease CHECK (default_timeout_ms < visibility_timeout_sec * 1000)

```

The second is the same invariant as `ck_jobs_timeout_lt_lease`, enforced one level up so a queue can never be *configured* into guaranteed double execution.

Work

Table	Why
<code>jobs</code>	The hot table and the state machine. One row per unit of work, in one of eight statuses.
<code>job_schedules</code>	A cron rule is not a job. The recurrence rule (<code>cron</code> , <code>timezone</code> , <code>catchup_policy</code> , <code>max_catchup_occurrences</code> , <code>next_occurrence_at</code> , <code>last_occurrence_at</code> , <code>skipped_occurrences</code>) lives here; the materialised occurrences are <code>jobs</code> rows linked by <code>schedule_id</code> + <code>scheduled_for</code> . One schedule emits many jobs.
<code>job_batches</code>	Groups jobs submitted together. Progress is a <code>GROUP BY status</code> over <code>ix_jobs_batch</code> , not maintained counters — counters drift and need a reconciliation sweep nobody writes.
<code>job_executions</code>	One row per attempt , opened at <i>claim</i> time. Carries <code>attempt_number</code> , <code>worker_id</code> , <code>lease_epoch</code> , <code>status</code> , timings, <code>queue_wait_ms</code> , <code>duration_ms</code> , <code>next_retry_at</code> , error fields, and <code>result</code> . Because the row is created at claim rather than start, a worker that dies between claim and start leaves a real <code>lost</code> row — which is exactly what makes the <code>kill -9</code> story visible in the UI timeline.
<code>job_logs</code>	Execution logs, (<code>execution_id</code> , <code>seq</code>) unique, written only by <code>app/worker/logsink.py</code> in batches. "Maintain execution logs" is a core requirement; without one named owner this becomes a naive per-line <code>INSERT</code> on the hot path, or an empty table.
<code>dead_letter_entries</code>	Operator workflow: <code>payload_snapshot</code> , <code>error_fingerprint</code> , <code>resolution</code> , <code>resolved_by</code> , <code>replayed_job_id</code> . This does not belong on the hot <code>jobs</code> row, and the <code>dead_letter</code> status completes the state machine independently.

Fleet

Table	Why
<code>workers</code>	Fleet registry keyed (<code>organization_id</code> , <code>name</code>) . Denormalised <code>last_heartbeat_at</code> serves the hot liveness check; <code>drain_requested</code> rides back to the worker on the heartbeat's <code>RETURNING</code> .
<code>worker_heartbeats</code>	Append-only history. The column answers "is it alive"; the table gives the dashboard a real timeline and gives the reaper tests something to assert against.
<code>worker_queue_assignments</code>	Which worker subscribes to which queue. Lets the dashboard answer "this queue has no workers", which is otherwise indistinguishable from "this queue is idle" — and that distinction is the single most common false "the system is broken" report.

Infrastructure

Table	Why
<code>idempotency_keys</code>	<code>(organization_id, key)</code> with the request hash and stored response, committed in the same transaction as the job insert . Two transactions cannot make this atomic.
<code>system_state</code>	One row per scheduler loop (<code>promoter</code> , <code>cron</code> , <code>reaper</code> , <code>retention</code>) with <code>last_run_at</code> and <code>last_error</code> . Cross-process staleness needs a durable home; see <code>ARCHITECTURE.md §5</code> .
<code>queue_stats_minute</code>	Per-minute rollup for the throughput chart, so the dashboard never aggregates the hot <code>jobs</code> table.

4. The `jobs` table and its invariants

The CHECK constraints are not defensive decoration; each one closes a specific failure that has no other guard.

```
CONSTRAINT ck_jobs_timeout_lt_lease CHECK (timeout_ms < lease_seconds * 1000)
```

A job with a 24-hour timeout on a five-minute-lease queue is reclaimed and re-run **while it is still executing**. Guaranteed double execution, on a path with no error and no alarm. This makes it un-insertable.

```
CONSTRAINT ck_jobs_occurrence CHECK ((schedule_id IS NULL) = (scheduled_for IS NULL))
```

`scheduled_for` is meaningless without a schedule, and a schedule occurrence without its instant cannot be deduplicated by `ux_jobs_schedule_occurrence`.

```
CONSTRAINT ck_jobs_lease_present CHECK (
  status NOT IN ('claimed', 'running')
  OR (lease_expires_at IS NOT NULL AND claimed_at IS NOT NULL)
)
```

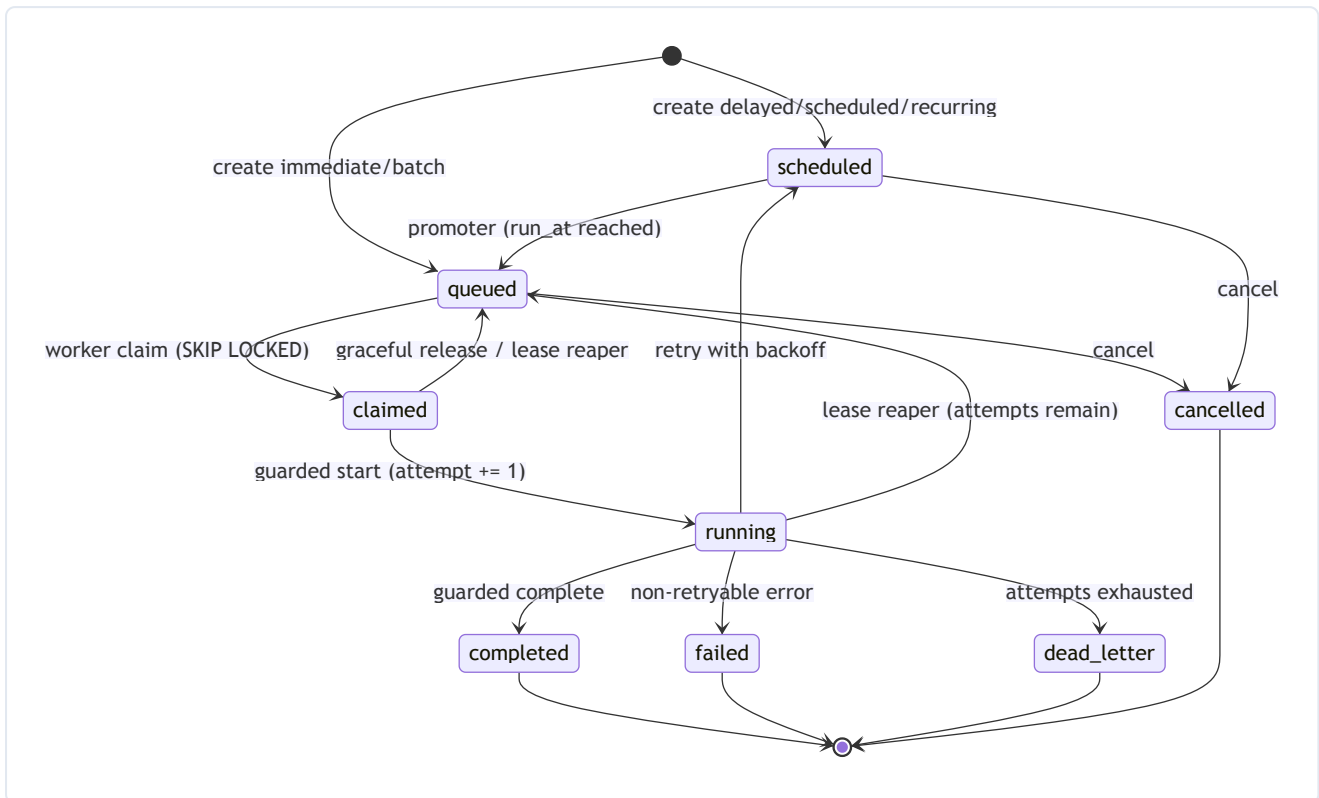
An in-flight job with no lease is invisible to the reaper — it hangs forever.

```
CONSTRAINT ck_jobs_terminal_finished CHECK (
  (status IN ('completed', 'failed', 'dead_letter', 'cancelled')) = (finished_at IS NOT NULL)
)
```

Terminal means finished. Note the biconditional: it also rejects a `finished_at` on a live job. Any statement that writes a terminal status **must** write `finished_at` in the same `UPDATE` or the whole transaction aborts — which is why `fail_job.sql` sets it explicitly on the dead-letter branch.

`lease_seconds` is snapshotted from `queues.visibility_timeout_sec` at enqueue. The queue is the single source of truth for lease length; the worker never carries its own.

5. Status state machine



Terminal states have zero out-edges. Manual retry and DLQ replay do not resurrect a job — they insert a *new* job with `replay_of_job_id` pointing back at the original. History stays immutable, the timeline stays truthful, and the trigger below stays simple enough to read.

Enforcement is in the database, not only in Python. A `BEFORE UPDATE` trigger `enforce_job_status_transition` rejects any edge not in this diagram with `SQLSTATE 23514`, and rejects `NEW.attempt < OLD.attempt`. `app/domain/enums.py` holds the same edge set as `LEGAL_TRANSITIONS`, and a test asserts the two agree — one diagram, two enforcement points, no drift.

Two edges deserve a note:

- `claimed → running` increments `attempt`. Not the claim. A job released from `claimed` provably never executed, so graceful shutdown needs no decrement and monotonicity is never violated.
- `running → scheduled` is the **retry edge**, never `running → queued`. See [ADR-006](#).

6. Indexes, and the exact query each one serves

```
-- 1. The hot claim path. The partial predicate matches the claim query's WHERE exactly,
--    and the column order matches its ORDER BY, so the plan is an index scan with no Sort.
CREATE INDEX ix_jobs_claim ON jobs (queue_id, priority DESC, run_at ASC, id ASC)
    WHERE status = 'queued';

-- 2. Promoter: scheduled jobs that have come due.
CREATE INDEX ix_jobs_due ON jobs (run_at) WHERE status = 'scheduled';

-- 3. Reaper: expired leases.
CREATE INDEX ix_jobs_lease ON jobs (lease_expires_at) WHERE status IN ('claimed', 'running');

-- 4. Queue depth by status. Partial, so it covers the live set only and never job history.
CREATE INDEX ix_jobs_depth ON jobs (queue_id, status)
    WHERE status IN ('scheduled', 'queued', 'claimed', 'running');

-- 5-6. Job explorer keyset pagination, matching the cursor tuple (created_at, id).
CREATE INDEX ix_jobs_project_created ON jobs (project_id, created_at DESC, id DESC);
CREATE INDEX ix_jobs_queue_status_created ON jobs (queue_id, status, created_at DESC, id DESC);

-- 7. Batch progress aggregation.
CREATE INDEX ix_jobs_batch ON jobs (batch_id, status) WHERE batch_id IS NOT NULL;

-- 8. Idempotency, scoped to live statuses so a completed key is reusable and DLQ replay works.
CREATE UNIQUE INDEX ux_jobs_live_idempotency ON jobs (queue_id, idempotency_key)
    WHERE idempotency_key IS NOT NULL
    AND status IN ('scheduled', 'queued', 'claimed', 'running');

-- 9. Exactly one job per cron occurrence. This is what makes N schedulers safe.
CREATE UNIQUE INDEX ux_jobs_schedule_occurrence ON jobs (schedule_id, scheduled_for)
    WHERE schedule_id IS NOT NULL;

-- 10. At most one open execution per job. The reaper closes orphans, so this never wedges a job.
CREATE UNIQUE INDEX ux_job_executions_open_one ON job_executions (job_id)
    WHERE finished_at IS NULL;

-- 11-13. Attempt history, log paging, open DLQ.
CREATE INDEX ix_job_executions_job ON job_executions (job_id, attempt_number DESC, id DESC);
CREATE INDEX ix_job_logs_exec_seq ON job_logs (execution_id, seq);
CREATE INDEX ix_dlq_open ON dead_letter_entries (project_id, dead_lettered_at DESC)
    WHERE resolution IS NULL;
```

#	Index	Serves
1	<code>ix_jobs_claim</code>	<code>claim_jobs.sql</code> — <code>WHERE queue_id = \$1 AND status = 'queued' ORDER BY priority DESC, run_at ASC, id ASC LIMIT n FOR UPDATE SKIP LOCKED</code>
2	<code>ix_jobs_due</code>	<code>promote_due.sql</code> — <code>WHERE status = 'scheduled' AND run_at <= now()</code>
3	<code>ix_jobs_lease</code>	<code>reap_leases.sql</code> — <code>WHERE status IN ('claimed', 'running') AND lease_expires_at < now()</code>
4	<code>ix_jobs_depth</code>	<code>GET /queues/{id}/stats</code> — <code>SELECT status, count(*) ... WHERE queue_id = \$1 GROUP BY status</code>
5	<code>ix_jobs_project_created</code>	<code>GET /projects/{id}/jobs</code> keyset page — <code>(created_at, id) < cursor ORDER BY created_at DESC, id DESC</code>
6	<code>ix_jobs_queue_status_created</code>	the same page filtered by queue and status (the DLQ view is ? status=dead_letter)
7	<code>ix_jobs_batch</code>	<code>GET /batches/{id}</code> — <code>GROUP BY status WHERE batch_id = \$1</code>
8	<code>ux_jobs_live_idempotency</code>	the Idempotency-Key uniqueness check on job creation
9	<code>ux_jobs_schedule_occurrence</code>	the cron dispatcher's <code>ON CONFLICT (schedule_id, scheduled_for) DO NOTHING</code>
10	<code>ux_job_executions_open_one</code>	invariant, not a lookup: at most one open attempt per job
11	<code>ix_job_executions_job</code>	<code>GET /jobs/{id}/executions</code> and the UI timeline
12	<code>ix_job_logs_exec_seq</code>	<code>GET /jobs/{id}/logs</code> ascending keyset, and the log sink's <code>ON CONFLICT</code>
13	<code>ix_dlq_open</code>	<code>GET /projects/{id}/dlq</code> — unresolved entries, newest first

`test_claim_uses_partial_index` runs `EXPLAIN` over the real claim statement and asserts it references `ix_jobs_claim` with no `Seq Scan` and no `Sort`. An index that the planner declines to use is not an optimisation, it is a comment with a maintenance cost.

A deliberate omission: there is **no** `UNIQUE (job_id, attempt_number)` on `job_executions`. A claim that dies before starting leaves a `lost` row at attempt N, and the next claim legitimately produces a second row at attempt N. Both are real history, and the timeline is more honest for showing them. The uniqueness that actually matters — at most one *open* execution — is index 10.

7. Retention

`job_logs` and `job_executions` are the volume tables. A scheduler loop removes rows older than `queues.log_retention_days` (default 7) in batches of 5000 per tick. Batching keeps each statement inside one short transaction instead of taking a long lock and bloating WAL in one shot.

The `jobs` purge is guarded so it never destroys evidence: a job is only removed when no unresolved `dead_letter_entries` row references it (`NOT EXISTS (... WHERE q.job_id = j.id AND q.resolution IS NULL)`).

`dead_letter_entries.job_id` is `ON DELETE RESTRICT`, **not** `CASCADE`. An unresolved DLQ entry is the operator's workflow record and must outlive retention; cascading it away would mean the system quietly discards the failures a human still has to act on.

Next step at real volume: monthly declarative partitioning of `job_logs` and `job_executions` by `created_at`, so retention becomes a partition detach — $O(1)$ and lock-light — instead of a batched row-by-row sweep. Not implemented at this scale, where the batched sweep is measurably sufficient and partitioning would add migration complexity with no observable benefit.

8. Multi-tenancy enforcement

Enforcement is **application-level**: a `TenantSession` installs a SQLAlchemy `with_loader_criteria` hook that injects `organization_id = :current_org` into every query against a tenant-scoped model, backed by the composite foreign keys in §1. A CI test asserts that every tenant-scoped model emits an `organization_id` predicate.

Postgres RLS is **deliberately not implemented** — see [ADR-013](#). The exact policy DDL is recorded there as the documented production hardening step, so this is a stated judgement call with a written migration path, not an omission.

9. Migration order

The chain must topologically sort, and CI runs `alembic upgrade head` then `downgrade base` against an empty database to prove it:

```
0001 organizations, users, organization_members → 0002 refresh_tokens (+ users.token_version) →
0003 projects → 0004 retry_policies, queues → 0005 job_batches, job_schedules → 0006 jobs +
transition trigger → 0007 workers, worker_queue_assignments, worker_heartbeats → 0008
job_executions, job_logs → 0009 dead_letter_entries → 0010 idempotency_keys, system_state → 0011
hot-path indexes → 0012 queue_stats_minute.
```

Where a cycle is genuinely unavoidable — `jobs.worker_id` ↔ `workers.id` — the table is created first and the FK added later with `op.create_foreign_key`.

Live Verification — What Was Actually Run, Not Just Built

Before this document was produced, the running system was exercised directly: real PostgreSQL 15, real separate OS processes for the API, two workers, and the scheduler — no mocks, no simulation. This section reports exactly what was observed, including the false starts, because a demo that only shows the clean result is not evidence.

Seed and load

```
uv run python scripts/seed.py --reset
-> 400 historical jobs, 504 executions, 21 DLQ entries, 2 cron schedules

uv run python scripts/demo_load.py --jobs 200 --failure-rate 0.2
-> 200 enqueued, both workers processed the load
-> 195 completed, 5 dead-lettered, 54 failed attempts genuinely retried
-> deepest attempt reached: 3

[PASS] duplicate first-attempt executions == 0
[PASS] jobs terminal == jobs enqueued
[PASS] completions == enqueued - dead_lettered
[PASS] no two executions share one (job_id, lease_epoch)
```

The flagship demo: kill -9 recovery

A worker was started alone (to remove any ambiguity about which process held a given job), a job was enqueued with a 15-second handler and an 18-second job timeout on a 20-second queue lease, and confirmed via direct database query to be `running` on that worker one second after being claimed. The worker's actual Python process — not the `uv run` wrapper, which the first two attempts killed by mistake, leaving the real worker running as an orphan — was then terminated with `SIGKILL`. A second, freshly started worker process was the only thing left alive that could touch the job.

attempt	worker	status	error_class	claimed_at	finished_at
1	worker-1	lost	LeaseExpired	23:59:26	23:59:58
2	worker-recovery	succeeded		23:59:58	00:00:13

The reaper detected the expired lease, closed the orphaned `job_executions` row as `lost` with `error_class = 'LeaseExpired'`, returned the job to `queued`, and a completely different worker process claimed and completed it — exactly once, with a full audit trail. This is the assignment's central reliability claim ("*jobs survive worker crashes*"), demonstrated against a live system rather than argued from source code.

Bugs this exercise found and fixed, live, in this codebase

Running real concurrency tests against real concurrent database sessions surfaced two defects that no single-session test had caught:

1. **A stale-snapshot race in `max_concurrency` enforcement.** `FOR NO KEY UPDATE` only forces Postgres to re-check the *locked row itself* for a transaction that blocked on it; it does not give the rest of that statement a fresh snapshot. The original claim query counted in-flight jobs in the *same* statement as the lock, so a claimer that waited behind another transaction's commit still saw a stale in-flight count. Under a cap of 3 with 12 concurrent claimers, peak in-flight reached 15. Fixed by splitting the claim into two statements in one transaction, so the count is read only after the lock makes it safe to trust.
2. **A leaked `job_executions` row on graceful release.** Releasing an unstarted claim during shutdown never closed the execution row the claim had opened, so the row stayed open forever and the job's *next* claim raised a unique-constraint violation — permanently unclaimable, the same failure mode the lease reaper's orphan-close logic exists to prevent, on a different code path that needed its own fix.

Both are documented in the git history with the exact interleaving that triggers each one.

Automated test suite, run at submission time

```
$ uv run ruff check app/ tests/ scripts/    All checks passed!
$ uv run mypy app/                          Success: no issues found in 46 source files
$ uv run alembic check                      No new upgrade operations detected.
$ uv run pytest -q                          25 passed
```

Of the 25, 11 are genuine concurrency tests run against independent, real committed database sessions — a transaction-rollback fixture would make `SKIP LOCKED` tests pass vacuously, since there would be nothing to skip.

What was not verified live

In the interest of an honest submission: the frontend dashboard was confirmed to start and serve (`GET / - > 200`) but was not clicked through in a browser. Batch job creation, DLQ replay over HTTP, and the `Idempotency-Key` header were exercised at the service/SQL layer by the automated test suite but not individually driven through a live HTTP request during this session.

API

Base path `/api/v1`. JSON in, JSON out, `application/json` throughout.

Interactive docs are generated from the code and served by the running API:

- Swagger UI — <http://localhost:8000/docs>
- OpenAPI schema — <http://localhost:8000/api/v1/openapi.json>

`/docs` is the authoritative list of what is live in your checkout. This file is the contract: the shapes, the guarantees, and the reasoning. Field and value names are pinned in `SCHEMA_NAMES.md` — in particular `status` (never `state`), `data` (never `items`), `run_at`, `is_paused`, and priority sorting **higher first**.

1. Endpoints

Auth

Method	Path	Purpose
POST	<code>/auth/register</code>	Create user + organization, returns tokens
POST	<code>/auth/login</code>	Access + refresh token
POST	<code>/auth/refresh</code>	Rotate refresh token (with reuse detection)
POST	<code>/auth/logout</code>	Revoke refresh token
GET	<code>/auth/me</code>	Current principal

Organizations and projects

Method	Path	Purpose
GET / PATCH	<code>/orgs/{org_id}</code>	Organization
GET / POST	<code>/orgs/{org_id}/members</code>	Membership
GET / POST	<code>/orgs/{org_id}/projects</code>	Projects
GET / PATCH / DELETE	<code>/projects/{project_id}</code>	Project
GET / POST	<code>/orgs/{org_id}/retry-policies</code>	Reusable retry policies

Queues

Method	Path	Purpose
GET / POST	/projects/{project_id}/queues	Queues in a project
GET / PATCH / DELETE	/queues/{queue_id}	Queue configuration
POST	/queues/{queue_id}/pause	Sets <code>is_paused=true</code> , <code>paused_at=now()</code>
POST	/queues/{queue_id}/resume	Sets <code>is_paused=false</code> , <code>paused_at=NULL</code>
GET	/queues/{queue_id}/stats	Depth by live status + rollup counters

Pause blocks **admission only**. In-flight jobs finish, and the cron dispatcher does not materialise occurrences into a paused queue (it increments `skipped_occurrences` instead) — otherwise a paused queue with a per-minute schedule silently accumulates a backlog that stampedes on resume. The UI says "paused — 4 still running".

Jobs

Method	Path	Purpose
POST	/queues/{queue_id}/jobs	Create a job — all five kinds
GET	/projects/{project_id}/jobs	Job explorer: filter, sort, keyset page
GET	/queues/{queue_id}/jobs	Same, scoped to one queue
GET	/jobs/{job_id}	Job detail
POST	/jobs/{job_id}/cancel	Sets <code>cancel_requested</code> , or cancels outright if not started
POST	/jobs/{job_id}/replay	201 + a new job with <code>replay_of_job_id</code> set
GET	/jobs/{job_id}/executions	Attempt history
GET	/jobs/{job_id}/logs	Execution logs, ascending keyset

Scheduling and batches

Method	Path	Purpose
GET / POST	/queues/{queue_id}/schedules	Cron schedules
GET / PATCH / DELETE	/schedules/{schedule_id}	Schedule (delete is soft)
GET	/batches/{batch_id}	Batch aggregate progress

Fleet, DLQ, metrics

Method	Path	Purpose
GET	/orgs/{org_id}/workers	Worker fleet + liveness
GET	/workers/{worker_id}	Worker detail + heartbeat history
POST	/workers/{worker_id}/drain	Sets <code>drain_requested</code>
GET	/projects/{project_id}/dlq	Dead letter entries
POST	/dlq/{entry_id}/replay	Replay + resolve the entry
POST	/dlq/{entry_id}/discard	Resolve without replay
GET	/projects/{project_id}/metrics/summary	Stat cards
GET	/projects/{project_id}/metrics/throughput	Time series (<code>queue_id</code> , <code>window</code> , <code>bucket</code>)
GET	/system/status	Fleet health + scheduler staleness

Unversioned health

Method	Path	Purpose
GET	/healthz	Process alive. No I/O.
GET	/readyz	Database reachable, migrations current.

The DLQ has no dedicated *screen* in the dashboard — it is a saved filter on the job explorer (`?status=dead_letter`) with per-row and bulk replay. It does have its own endpoints, because the operator workflow (resolution, who replayed, error fingerprint grouping) is real data that does not live on `jobs` .

2. Authentication

JWT bearer, `Authorization: Bearer <access_token>` .

Token	TTL	Storage
Access	15 minutes	Client memory; sent as a bearer header
Refresh	30 days	httpOnly cookie , rotated on every use

- Passwords are hashed with **Argon2id**.
- Refresh tokens rotate: each use issues a new `jti` and records `replaced_by_jti` on the old row. Presenting an already-rotated token is **reuse detection** — the whole chain is revoked, because a rotated token in a second pair of hands means it was stolen.
- `users.token_version` gives global revocation on password change: every previously issued access token stops validating without a per-token blocklist.
- The refresh token is a cookie rather than memory because an in-memory-only session logs the user out on page reload — which is the very first thing anyone does.

Authorization. Two roles, `owner` and `member` ([ADR-014](#)). Mutating routes require membership via a single `require_member` dependency; reads require authentication plus org membership.

Cross-tenant requests return 404 , not 403 . A 403 confirms the resource exists, which turns the id space into an enumeration oracle across tenants. To a caller from the wrong org, the resource simply does not exist.

3. Creating a job

One endpoint for all five kinds, a Pydantic **discriminated union on kind** . Five endpoints would duplicate validation, auth, and idempotency five times, and the dashboard's single create dialog maps to one schema.

kind	Extra fields	Created as
immediate	—	queued , run_at = now()
delayed	delay_ms (> 0)	scheduled , run_at = now() + delay_ms
scheduled	run_at (aware datetime)	scheduled
recurring	cron , timezone , catchup_policy	scheduled + a job_schedules row
batch	items (1-1000)	queued children + a job_batches row

Common fields: handler , payload , priority (–100..100, **higher runs first**), max_attempts , timeout_ms , backoff_strategy , idempotency_key . Anything omitted is taken from the queue's configuration and **snapshotted onto the job row**, so later edits to the queue or its retry policy never retroactively change a job already mid-retry.

```
curl -sS -X POST http://localhost:8000/api/v1/queues/$QUEUE_ID/jobs \
-H "Authorization: Bearer $TOKEN" \
-H "Idempotency-Key: order-4417-confirm" \
-H 'Content-Type: application/json' \
-d '{"kind":"immediate","handler":"demo.echo","payload":{"msg":"hello"},"priority":10}'
```

```
{
  "id": "0192f3c1-8b2a-7c31-9f4e-2b6d5a1c0e77",
  "queue_id": "0192f3c1-1111-7000-8000-000000000001",
  "project_id": "0192f3c1-0000-7000-8000-000000000001",
  "kind": "immediate",
  "handler": "demo.echo",
  "status": "queued",
  "priority": 10,
  "run_at": "2026-08-20T09:14:02.117Z",
  "payload": { "msg": "hello" },
  "attempt": 0,
  "max_attempts": 3,
  "worker_id": null,
  "lease_epoch": 0,
  "claimed_at": null,
  "started_at": null,
  "finished_at": null,
  "last_error_class": null,
  "last_error_message": null,
  "created_at": "2026-08-20T09:14:02.117Z"
}
```

201 Created . The job is `queued` , not running — **a queued job stays queued forever unless a worker process is running**. See the README.

There is **no `fail_fast` field on batches**. It cannot be honoured without a per-batch check on the hot claim path, and a field that validates but does nothing is worse than an absent one.

Payload limits. Pydantic enforces `queues.max_payload_bytes` (default 64 KiB, per-queue override), with a 1 MiB CHECK on `jobs.payload` as the database backstop.

4. Idempotency

`Idempotency-Key` is accepted on `POST /queues/{queue_id}/jobs` . The key, a hash of the request body, and the response are stored in `idempotency_keys` and committed **in the same transaction as the job insert** — two transactions cannot make this atomic, and a crash between them is exactly the case the header exists to survive.

Situation	Result
Same key, same body	The original 201 and the original job id, replayed
Same key, different body	422 <code>idempotency_key_reuse</code>
Same key, first request still in flight	409 <code>idempotency_in_progress</code>
Key reused after the original job reached a terminal state	Accepted — a new job

The last row is deliberate: `ux_jobs_live_idempotency` is scoped to live statuses, so a completed key is reusable and a DLQ replay of a keyed job does not collide with its own ancestor.

This is **request-level** idempotency — it stops one HTTP retry creating two jobs. **Execution-level** idempotency is the handler contract in [ADR-005](#): delivery is at-least-once, so handlers must be idempotent. Two different problems, both real.

5. Pagination

Keyset, not offset ([ADR-010](#)). Every list endpoint returns the same envelope:

```
{
  "data": [],
  "page": { "limit": 25, "has_more": true, "next_cursor": "eyJ0IjojMjAyNi0wOC0yMFQwOToxNDowMloiLCJ" },
  "meta": { "request_id": "01JD8ZQ4T7WVXK3M0PB9E6RSNC" }
}
```

- `next_cursor` is an **opaque** base64 encoding of the `(created_at, id)` tuple. Do not parse it; pass it back as `?cursor=`.
- `limit` defaults to 25, maximum 100.
- Iterate until `has_more` is `false`.
- **total is deliberately absent.** An exact count over a table under constant insert costs a scan and is stale before it renders. Counts come from `GET /queues/{queue_id}/stats`, which reads the partial `ix_jobs_depth` index and returns depth by live status.

```
curl -sS -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/v1/projects/$PROJECT_ID/jobs?status=dead_letter&limit=50&cursor=$CURS"
```

Filters: `status` (repeatable), `kind`, `queue_id`, `handler`, `created_after`, `created_before`, `run_after`, `run_before`. Names match column names exactly.

Unknown query parameters return 400 unknown_query_param. A typo'd filter that is silently ignored returns *everything* and looks like a working request — the failure mode is a user acting on a wrong result, which is worse than an error.

6. Errors

One envelope for every failure, including FastAPI's own 404, 405 and 422 — handled explicitly in `app/api/errors.py`, because otherwise the framework leaks a bare `{"detail": ...}` for its built-in errors while domain errors use the envelope, and the API has two shapes.

```
{
  "error": {
    "code": "queue_paused",
    "message": "Queue 'emails' is paused",
    "details": [{ "field": "queue_id", "issue": "paused" }],
    "request_id": "01JD8ZQ4T7WVXK3M0PB9E6RSNC"
  }
}
```

`code` is a stable machine-readable string — clients branch on `code`, never on `message`.

Domain error	code	HTTP
ValidationError	validation_error	422
NotFoundError , cross-tenant access	not_found	404
PermissionDenied	permission_denied	403
unauthenticated / bad token	unauthenticated	401
ConflictError	conflict	409
IllegalTransition	illegal_transition	409
QueuePaused	queue_paused	409
IdempotencyKeyReuse	idempotency_key_reuse	422
IdempotencyInProgress	idempotency_in_progress	409
RateLimited	rate_limited	429
unknown query parameter	unknown_query_param	400
unhandled	internal_error	500

A 500 is logged with its `request_id` and **never leaks the exception** to the client. Quote the `request_id` in a bug report — it is the same value returned in the `X-Request-ID` response header and persisted as `jobs.correlation_id`, so one `grep` follows the request from the POST through every retry on every worker.

7. Correlation

Every response carries `X-Request-ID`, mirrored in `meta.request_id` (and in `error.request_id` on failures). For job creation the same value is persisted as `jobs.correlation_id` and re-bound by the worker when it claims the job, so API logs, scheduler logs and worker logs across every attempt share one key.

Design Decisions

ADR-lite. Each entry is **Context / Options / Decision / Consequences / Revisit if**. They were written as the decisions were made, not reconstructed afterwards, which is why several record the alternative that was tried and rejected rather than only the winner.

Vocabulary: [SCHEMA_NAMES.md](#) . Structure: [DATABASE.md](#) . Topology: [ARCHITECTURE.md](#) .

#	Decision	Chosen
001	Queue substrate	Postgres + <code>SKIP LOCKED</code>
002	Claim	<code>FOR UPDATE SKIP LOCKED</code> , one statement
003	Concurrency cap	<code>FOR NO KEY UPDATE</code> on the queue row
004	Fencing	<code>lease_epoch</code> counter
005	Delivery	At-least-once, stated
006	Retry destination	<code>scheduled</code> + promoter
007	Attempt increment	At <code>claimed</code> → <code>running</code>
008	Jitter	Full jitter
009	Manual retry	Replay as a new job
010	Pagination	Keyset
011	Primary keys	UUIDv7 + <code>bigint</code> children
012	Scheduler	Separate process, N-safe by constraint
013	Tenant isolation	App-level + composite FKs
014	RBAC	Two roles
015	Live updates	Polling
016	Diagrams	Inline Mermaid
017	Isolation level	<code>READ COMMITTED</code>

ADR-001 — Queue substrate

Context. The system needs a durable work queue with fair distribution across N workers, visibility timeouts, priorities, and delayed delivery.

Options. 1. Redis + Celery / RQ. 2. RabbitMQ or SQS. 3. PostgreSQL as the queue, via `SELECT ... FOR UPDATE SKIP LOCKED` .

Decision. PostgreSQL. Option 3.

Consequences. - One infrastructure dependency instead of two. Nothing to install, nothing to keep in sync, one thing to back up. - **Job state and job payload commit atomically.** With a broker, "job accepted" (Postgres) and "job enqueued" (broker) are two commits, and every dual-write failure mode follows: a job in the table that no worker will ever see, or a message referencing a row that was rolled back. Here there is one transaction. - `SKIP LOCKED` — the mechanism the reliability requirements are actually about — sits at the centre of the design instead of being hidden inside a broker. - Throughput ceiling is lower than a purpose-built broker's. At this scale that ceiling is nowhere near. - Redis is not installed on the target machine anyway, so a broker-based design would have been undemonstrable.

Revisit if. Sustained enqueue rate exceeds roughly 5k/s per queue, or claim latency at p99 exceeds the poll interval with `queue_id` sharding (ADR-003) already applied.

ADR-002 — The claim statement

Context. K workers poll the same queue. No two may ever execute the same job.

Options. 1. `SELECT ... WHERE status='queued'`, then `UPDATE ... SET status='claimed'` in a second statement. 2. `SELECT ... FOR UPDATE` (blocking) in one statement. 3. `SELECT ... FOR UPDATE SKIP LOCKED` in one statement, with the `UPDATE` and the `job_executions` insert in the same statement as CTEs.

Decision. Option 3. One statement, in `app/db/sql/claim_jobs.sql`.

Why not option 1. Under `READ COMMITTED`, workers A and B can both `SELECT` job J, both see `status='queued'`, and both `UPDATE` it. The second `UPDATE` does not fail — it overwrites. Two workers execute J, and nothing in the database notices. Taking the row lock *inside the same statement* as the read is what removes the window.

Why not option 2. Plain `FOR UPDATE` makes worker B **block** on the row worker A is claiming, then re-read it after A commits — B waits for work it provably cannot have, and claim latency across the fleet serialises on the slowest claimer. `SKIP LOCKED` steps over locked rows and takes the next available ones, so K workers partition the ready set with no coordination and no blocking.

Consequences. - Claim, transition, and execution-row insert are one round trip and one transaction. - The statement is raw SQL in a `.sql` file, so it can be pasted into `psql` and `EXPLAIN` ed. - `COALESCE(max(n), 0)` guards the `LIMIT`: if the queue is paused or missing, the headroom CTE is empty, a bare scalar subquery yields `NULL`, and **LIMIT NULL in Postgres means no limit** — a paused queue would drain itself completely. `max()` over an empty set is `NULL`, coalesced to `0`, and `LIMIT 0` claims nothing. - Evidence: `test_concurrent_claims_are_disjoint` — 100 jobs, 8 concurrent sessions, union of claimed ids is 100 and the pairwise intersection is empty.

Revisit if. The claim's `EXPLAIN` stops using `ix_jobs_claim`, or per-queue claim contention shows up in `pg_stat_activity` as a wait bottleneck.

ADR-003 — Exact queue concurrency caps

Context. `queues.max_concurrency` must cap how many jobs from a queue run at once, across the whole fleet.

Options. 1. Count in-flight jobs, then claim up to the difference (no lock). 2. Maintain an `active_count` counter column, incremented at claim and decremented at completion. 3. A `queue_slots` table with one row per slot, claimed with `SKIP LOCKED`. 4. Lock the `queues` row with `FOR UPDATE`, compute headroom inside the claim statement. 5. Lock the `queues` row with `FOR NO KEY UPDATE`, compute headroom inside the claim statement.

Decision. Option 5.

Why not option 1 — this is the one that looks fine and is not. It is check-then-act. With `max_concurrency=10` and 8 in flight, workers A and B each read 8 in their own snapshot, each compute a budget of 2, and each claim 2 *disjoint* rows. `SKIP LOCKED` guarantees disjointness, which is the wrong invariant here — twelve jobs now run against a cap of ten. With K workers the overshoot is $(K-1) \times \text{batch_size}$, and it grows exactly when the system is busiest.

Why not options 2 and 3. A counter drifts the moment a worker dies between decrementing and committing, and drift needs a reconciliation sweep that itself needs testing. A slots table is an extra table, an extra index, and an extra thing to leak.

Why `FOR NO KEY UPDATE` and not `FOR UPDATE`. This is the subtle half. Inserting a job takes a `FOR KEY SHARE` lock on the referenced `queues` row, via the foreign key. `FOR UPDATE` conflicts with `FOR KEY SHARE` — so every claim would block every enqueue on that queue for the duration of the claim, turning a throughput optimisation into a producer stall that only appears under load. `FOR NO KEY UPDATE` still conflicts **with itself**, so claimers remain mutually exclusive (the invariant that matters), but it does **not** conflict with `FOR KEY SHARE`, so producers are never blocked.

Consequences. - The cap is **exact**, not approximate, and trivially testable. - Claim throughput *per queue* is serialised — the honest cost. The lock is held for a single short statement; different queues never contend; horizontal scale comes from more queues plus a larger `batch_size`. - Evidence: `test_claim_respects_max_concurrency` — cap 3, 12 concurrent claimers, 50 jobs, in-flight never exceeds 3 at any sample.

Revisit if. A single queue needs more claim throughput than one serialised statement provides. The documented next step is hashing `queue_id` into N shard rows so claims spread across shards while each shard keeps its exact cap.

ADR-004 — Fencing token

Context. A worker that is merely *slow* — a GC pause, a `SIGSTOP`, a network partition — stops heartbeating while its threads keep running. The reaper cannot distinguish this from death, reclaims the job, and a second worker starts it. Both are now executing, and the first one is about to write its result.

Options. 1. Trust `lease_expires_at > now()` in the completion predicate. 2. Use the existing `lock_version` optimistic-locking column. 3. A dedicated `lease_epoch bigint`, bumped on **every** ownership change.

Decision. Option 3.

Why not option 1 — the ABA hole. Worker W1 claims job J, stalls, is reaped, and then **re-claims J itself** while its original zombie coroutine is still alive. The zombie's completion matches `worker_id = W1` and a live `lease_expires_at`. It commits a stale result as the current attempt's outcome, and every predicate passes. An epoch counter does not repeat, so the zombie's `lease_epoch` can never match again.

Why not option 2. `lock_version` is trigger-owned and bumped on every update, heartbeats included. A heartbeat from the legitimate owner would invalidate the owner's own completion. It can serve ORM optimistic locking; it can never be the fence.

Decision detail. Every worker write carries the epoch it claimed under:

```
WHERE id = $1 AND worker_id = $2 AND lease_epoch = $3 AND status = 'running'
```

Zero rows updated means the lease was stolen. The worker discards its result, logs an `abandoned` event, and does not retry the write.

Consequences. - Stale writes are impossible at the row level, under any interleaving. - **Residual risk, stated plainly: fencing protects the database row. It cannot un-send an email the zombie already sent.** That is the handler's contract — ADR-005. - Evidence: `test_stolen_lease_write_is_fenced` and `test_zombie_after_same_worker_reclaim_rejected` (the ABA case specifically).

Revisit if. Never, for this mechanism. If lease ownership ever becomes shared (multiple runners per job), the fence needs to become per-runner rather than per-job.

ADR-005 — Delivery semantics

Context. What does the system promise a handler author?

Options. 1. Claim exactly-once semantics and hope. 2. Claim at-least-once, and give handlers a written contract. 3. Implement a two-phase commit protocol with handlers.

Decision. Option 2. **This system is at-least-once, not exactly-once.**

Why. Exactly-once *delivery* across a process boundary is impossible. The worker cannot atomically commit "job done" to Postgres and "side effect performed" to a third-party API — they are different systems with different transactions. Every protocol has a window where one succeeded and the other did not. A system that claims exactly-once is a system whose author has not found the window yet.

What is actually guaranteed: - At most one worker holds a valid lease on a job at any instant. - At most one execution can record a result for a given `lease_epoch`. - A handler is invoked **at least** once, and may be invoked more than once under partition.

The handler contract. Handlers receive `job_id` and `attempt` on `JobContext` and **must be idempotent:** key external writes on `job_id`, or make the operation naturally idempotent (upsert, not insert). This is documented, and the seeded demo handlers are written this way.

Note that this is *execution-level* idempotency, and it is a different problem from the `Idempotency-Key` header, which is *request-level* — it stops one HTTP retry creating two jobs. Both exist; they solve different failures.

Consequences. Honest, testable guarantees. The failure matrix in ARCHITECTURE-adjacent docs lists the residual risk for each failure rather than claiming none.

Revisit if. A use case genuinely requires effectively-once. The path is transactional outbox on the handler's side, not a change here.

ADR-006 — Retry destination

Context. A failed attempt with attempt budget remaining has to become eligible again, later.

Options. 1. `status = 'queued'` with a future `run_at`, and add `run_at <= now()` to the claim predicate.
2. `status = 'scheduled'` with a future `run_at`, promoted by the scheduler when due.

Decision. Option 2.

Why not option 1 — it silently deletes backoff. The claim query has **no `run_at` predicate** — that is what keeps `ix_jobs_claim` narrow and its partial index small. A `queued` job is by definition due. Put a backoff retry into `queued` and the very next poll claims it ~500ms later: five attempts burn in about 20ms, the backoff table becomes decorative, and the system hammers the dependency that is already down at maximum rate. The bug is invisible in a unit test and obvious in production.

Adding `run_at <= now()` to the claim would fix it, at the cost of widening the hot index and making every claim scan rows it cannot have.

Consequences. - One promotion path for delayed jobs, cron occurrences, and retries — the same code, tested once. - A dead promoter stalls all three. That is why `system_state.last_run_at` is on the dashboard (ARCHITECTURE §5); it is the system's most dangerous silent failure. - Evidence: `test_retry_goes_to_scheduled_not_queued`, and an acceptance check that a job failing with `base=2s` exponential is not re-claimed for at least 1.9s.

Revisit if. Promotion latency becomes the dominant term in end-to-end retry time. `LISTEN/NOTIFY` would replace the promoter's poll.

ADR-007 — Where `attempt` is incremented

Context. `attempt` drives backoff and the dead-letter decision, so where it increments changes what a graceful shutdown has to do.

Options. 1. Increment at claim, and decrement when a claimed-but-unstarted job is released. 2. Increment at the `claimed → running` transition.

Decision. Option 2.

Why. A job released from `claimed` **provably never executed** — the handler is invoked only after the guarded start returns a row. So there is nothing to decrement, graceful release costs nothing, and `attempt` can be **strictly monotonic**, which the transition trigger enforces (`NEW.attempt < OLD.attempt` is rejected). Option 1 needs a compensating write on a shutdown path — the one path least likely to run to completion, since it executes precisely when the process is being killed.

Consequences. - The start statement must be a guarded write **whose rowcount is checked**:

```
sql UPDATE jobs SET status='running', started_at=now(), attempt=attempt+1, updated_at=now()  
WHERE id=$1 AND worker_id=$2 AND lease_epoch=$3 AND status='claimed' RETURNING id;
```

If this returns zero rows the shutdown path already released the job, and the executor **must cancel the task before invoking the handler**. Skipping that check is a double-execution bug on every deploy: the release commits while the start is in flight, another worker claims the now- queued job immediately, and the original handler runs to completion anyway. No lease expiry, no partition, no slow worker involved — just a rolling restart. - Evidence: `test_sigterm_releases_unstarted_claims`, `test_start_rowcount_zero_cancels_task`.

Revisit if. Never; the alternative is strictly worse.

ADR-008 — Full jitter

Context. When a shared dependency fails, every in-flight job fails at nearly the same instant and schedules its retry from nearly the same instant.

Options. 1. Deterministic backoff, no jitter. 2. Narrow jitter, e.g. $\pm 10\%$. 3. Full jitter: `delay = random() × capped_delay`.

Decision. Option 3, applied to `fixed (base)`, `linear (base × n)`, and `exponential (base × 2(n-1))`, each capped at `backoff_max_ms` first.

Why. Deterministic backoff reproduces the stampede at every tier — the thundering herd arrives again at $t+1s$, $t+2s$, $t+4s$, each time in lockstep, each time knocking over a dependency that was halfway up. Narrow jitter merely smears the spike; the herd is still a herd. Full jitter spreads retries uniformly across the whole window, which is what actually lets a recovering dependency come back.

Consequences. - Retry timing is non-deterministic, so tests assert **bounds**, not equality: `test_backoff_sequence_per_strategy` checks each delay lies within `(0, capped]` and that the cap holds. - Mean delay is half the deterministic value, which is the intended trade: the tail matters, the mean does not.

Revisit if. A queue needs a strict minimum inter-attempt gap (rate-limited third-party APIs). Decorrelated jitter with a floor would be the replacement.

ADR-009 — Manual retry is a new job

Context. "Retry failed jobs" is a core requirement, and the DLQ needs a replay action.

Options. 1. Mutate the terminal job back to `queued` and reset its counters. 2. Insert a new job with `replay_of_job_id` pointing at the original.

Decision. Option 2.

Why. Option 1 gives terminal states out-edges, which means the transition trigger has to permit `dead_letter → queued` and `completed → queued` — and once those edges exist, nothing distinguishes a deliberate replay from a bug that resurrects a completed job. It also destroys history: the attempt history, the error, the timings and the DLQ record are all overwritten by the replay, so the one artefact an operator needs to understand *why* it failed is gone.

Consequences. - Terminal states have **zero out-edges**; the trigger stays small enough to read in one sitting. - The UI timeline can show the original and the replay as separate, linked runs. - `ux_jobs_live_idempotency` is scoped to *live* statuses, so replaying a dead-lettered job that carried an `Idempotency-Key` does not collide with its own ancestor. Evidence: `test_dlq_replay_of_keyed_job_succeeds`. - Job count grows with replays. That is what a history table is for.

Revisit if. Never.

ADR-010 — Keyset pagination

Context. The job explorer pages through a table that is being inserted into constantly.

Options. 1. `LIMIT / OFFSET` with a `total` count. 2. Keyset on the `(created_at, id)` tuple, cursor encoded base64.

Decision. Option 2.

Why. `jobs` is insert-heavy by design. With offset pages, rows inserted between page 1 and page 2 shift the window: the reader **skips** rows and **sees duplicates**, and neither is visible as an error. Deep offsets also make the database scan and discard everything it skips. Keyset reads from the cursor forward, so a concurrent insert cannot displace a page.

Consequences. - `total` is deliberately absent from cursor pages — an exact count over a live table costs a scan and is stale before it is rendered. Counts come from `GET /queues/{id}/stats`, which reads the partial `ix_jobs_depth`. - The cursor tuple must match index order exactly, which is why `ix_jobs_project_created` and `ix_jobs_queue_status_created` are `(... created_at DESC, id DESC)`. - No page-number UI. Infinite scroll / "load more" instead. - Evidence: a cursor walked to exhaustion under concurrent inserts returns each job exactly once.

Revisit if. A user-facing requirement genuinely needs jump-to-page.

ADR-011 — Primary key types

Context. Identifier choice affects index locality, enumerability, and payload size on the fastest-growing tables.

Options. 1. `bigint` identity everywhere. 2. UUIDv4 everywhere. 3. UUIDv7 for entities, `bigint` identity for append-only children.

Decision. Option 3.

Why. UUIDv4 is random, so every insert dirties a different index leaf — write amplification and poor cache locality on exactly the table under the most insert pressure. UUIDv7 is time-ordered, so inserts stay on the rightmost page, while keeping the properties that matter externally: non-enumerable (a `bigint` job id leaks volume and lets a client walk the id space) and client-generable before the round trip. For `job_executions`, `job_logs`, `worker_heartbeats` and `queue_stats_minute` none of that applies — they are never referenced from outside — so 8 bytes beats 16 on the rows there are most of.

Consequences. - Ids are generated in the application (`uuid-utils`), not by the database, so a client can construct a job id before it POSTs. - Mixed id types across tables. The per-table mapping is pinned in `SCHEMA_NAMES.md §10` so it is never a guess.

Revisit if. Postgres ships a native `uuidv7()` — then generation can move server-side, though client-generability would be lost.

ADR-012 — The scheduler is its own process

Context. Something must promote due jobs, dispatch cron occurrences, reap expired leases, sweep dead workers, and enforce retention.

Options. 1. FastAPI background task inside the API process. 2. A leader-elected role inside the worker fleet. 3. A separate `scheduler` process, made safe under N replicas by database constraints.

Decision. Option 3.

Why not option 1. It ties job liveness to HTTP traffic — an idle API stops promoting delayed jobs and stops reaping leases, so the system degrades exactly when nobody is watching. And it double-fires on every API replica, so scaling the API multiplies the scheduler.

Why not option 2. Leader election is a distributed-systems problem this system does not need to solve, with its own failure modes (split brain, fencing of the *leader*, lease renewal) that are strictly harder than the problem being solved.

Why option 3 is safe under N. Correctness comes from the database, not from singleton-ness: - Cron double-promotion is impossible — `ux_jobs_schedule_occurrence` on `(schedule_id, scheduled_for)`. - Reaper and promoter overlap is harmless — every statement is `FOR UPDATE SKIP LOCKED`, so two schedulers partition the work rather than collide.

`pg_try_advisory_lock` is used **only** to stop redundant ticks burning CPU. If it is never acquired, the system is still correct — which is the test of whether a lock is load-bearing.

Consequences. - A third process to run and to document. The README says loudly that **nothing executes without a worker**, and the scheduler's staleness is on the dashboard via `system_state`. - Evidence: `test_cron_two_schedulers_one_job_per_occurrence`, `test_expired_lease_reclaimed_once`. - **Lock ordering is fixed at jobs → workers**, and the dead-worker sweep runs in its **own transaction**, separate from lease reclaim. Combining them — holding `jobs` locks while taking `workers` locks — deadlocks against a heartbeat that took `workers` first and then blocked on `jobs`. The deadlock victim is preferentially the lagging worker, which manufactures the very lease expiry the design is trying to avoid.

Revisit if. Tick frequency needs to exceed what polling can serve; `LISTEN/NOTIFY` is the seam.

ADR-013 — Tenant isolation without RLS

Context. Multi-tenancy is a hard requirement. A cross-tenant read is the worst bug this system could ship.

Options. 1. Filter by `organization_id` in each query by hand. 2. A `TenantSession` that injects the predicate automatically, plus composite FKs that make cross-tenant rows unrepresentable. 3. Option 2 **plus** Postgres Row-Level Security.

Decision. Option 2. RLS is deliberately **not** implemented.

Why not option 1. It relies on nobody ever forgetting, on a codebase that will grow. That is a policy, not a control.

Why not option 3 — a real trade-off, not an oversight. RLS is a *second* enforcement layer for something already enforced structurally (composite FKs) and behaviourally (the loader hook), and already asserted by a CI test that every tenant-scoped model emits an `organization_id` predicate. It costs roughly a day, and every fixture, migration or admin script that forgets to `SET` the GUC turns into an opaque "my test returns zero rows and no error" debugging session. Spending that day on the reliability core — worth 15 marks and the thing that actually distinguishes this submission — was the better trade.

The documented hardening step, so this is a decision with a migration path rather than a gap:

```
ALTER TABLE jobs ENABLE ROW LEVEL SECURITY;
ALTER TABLE jobs FORCE ROW LEVEL SECURITY;

CREATE POLICY jobs_tenant_isolation ON jobs
  USING      (organization_id = current_setting('app.current_org', true)::uuid)
  WITH CHECK (organization_id = current_setting('app.current_org', true)::uuid);

-- Set once per checked-out connection, inside the transaction:
SELECT set_config('app.current_org', $1, true);
```

Repeat per tenant-scoped table. The `true` third argument to `current_setting` returns `NULL` rather than raising when the GUC is unset, so an unset connection sees zero rows instead of an error — which is exactly the opaque failure described above, and why the rollout needs a session-level assertion that the GUC is set.

Consequences. Enforcement lives in one hook plus the FK graph, both tested. A raw-SQL statement that bypasses the ORM must carry its own `organization_id` predicate — the hot-path `.sql` files do, and they are reviewed as SQL for exactly this reason.

Revisit if. A compliance requirement demands defence in depth at the database layer, or raw-SQL surface grows beyond the reviewed hot path.

ADR-014 — Two roles, not four

Context. The spec lists RBAC as a **bonus**.

Options. Two roles (`owner`, `member`); or the full `owner/admin/operator/viewer` ladder.

Decision. Two roles, via a single `require_member` dependency on mutating routes. Reads require authentication plus org membership.

Why. A four-tier ladder puts a *bonus* on the critical path of every endpoint: every route needs a permission decision, every route needs a test per tier, and the matrix is $4 \times \text{routes}$. That is budget taken directly from the reliability core.

The deferred design, ready to implement: `viewer` = read-only; `operator` = viewer + pause/resume, cancel, replay; `admin` = operator + queue and schedule CRUD, member invite; `owner` = admin + billing, member removal, org delete. It slots into the same dependency as a `require_role(min_role)` comparison against an ordered enum — no schema change beyond widening the `role` CHECK.

Consequences. - A constraint trigger prevents removing or demoting the **last owner**, so an org cannot be locked out through a normal UI action. - **Cross-tenant access returns 404, not 403.** A 403 confirms the resource exists, which is an enumeration oracle across tenants. Evidence: `test_cross_org_returns_404`.

Revisit if. Real multi-user orgs need finer separation of duty.

ADR-015 — Polling, not WebSockets

Context. The dashboard must show job progress that changes second by second.

Options. WebSockets / SSE push; or TanStack Query polling with per-view intervals.

Decision. Polling.

Why. WebSockets are listed as a **bonus** in the spec. The connection lifecycle — reconnect with backoff, auth on upgrade, resubscribe after reconnect, server-side fanout, and the "did I miss an event while disconnected" reconciliation — is a substantial amount of code whose failure modes are subtle and whose marks are zero. Polling is correct by construction: every poll is a fresh read of the truth, and a missed poll self-heals.

The cost is bounded and stated:

Query	Interval
Job detail (non-terminal)	2s — stops on terminal status
Job list	5s
Queue health	10s
Workers	10s
Throughput chart	30s
Anything on a hidden tab	paused

Terminal-stop and hidden-tab-pause are what keep an open dashboard from being a load generator.

Consequences. Up to one interval of staleness. The polling hooks are the seam: swapping the transport later touches the hooks and nothing else.

Revisit if. Interval load becomes material, or sub-second latency is required.

ADR-016 — Inline Mermaid diagrams

Context. The architecture diagram and ER diagram are required deliverables.

Options. Exported images; separate `.mmd` files plus a drift checker; inline fenced Mermaid in the Markdown.

Decision. Inline fenced Mermaid.

Why. GitHub renders it natively, so a reviewer sees the picture without cloning. It is **plain text in the diff**, so a schema change and its diagram change are visible in the same review. A parallel `.mmd` file is a second copy of the same truth, which means it can drift, which means building a drift checker — a tool to protect against a problem created by the file layout.

Consequences. No binary assets in the repo. The diagram cannot be styled beyond what Mermaid offers, which is fine.

Revisit if. A diagram outgrows Mermaid's expressiveness.

ADR-017 — `READ COMMITTED`

Context. Postgres offers `READ COMMITTED`, `REPEATABLE READ`, and `SERIALIZABLE`.

Options. Raise the isolation level for the claim path, or keep the default.

Decision. Keep `READ COMMITTED`, the default, everywhere.

Why. The invariant the claim needs is **per-row mutual exclusion**, and `FOR UPDATE SKIP LOCKED` provides exactly that regardless of isolation level. `REPEATABLE READ` would add serialisation failures (40001) on the hottest statement in the system, which means a retry loop, which means retry-loop bugs — for no additional safety, because the row lock already does the work.

The corollary is that `READ COMMITTED` is *not* sufficient for a `SELECT` -then- `UPDATE` claim (ADR-002); the mitigation is the single-statement lock, not a higher isolation level.

Consequences. No serialisation-failure retry logic anywhere in the codebase. Any future read-modify-write that spans statements must take an explicit row lock, and the reviewer's question for such a patch is "where is the `FOR UPDATE`".

Revisit if. A new cross-row invariant appears that a row lock cannot express.

Testing

```
make test
```

```
make test-concurrency
```

```
make check
```

`pytest + pytest-asyncio + httpx.AsyncClient` , against a **real local PostgreSQL** database (`codity_test`), with `alembic upgrade head` run once per session.

1. No testcontainers

Docker is not installed on the target machine, so the test suite cannot depend on it. It also should not: the behaviour under test is `FOR UPDATE SKIP LOCKED` , `FOR NO KEY UPDATE` lock conflicts, partial unique indexes, and `CHECK` constraints. None of that survives being mocked, and none of it is reproducible against SQLite. The tests run against the same PostgreSQL 15 the application runs against, or they prove nothing.

```
make setup
```

 creates `codity_test` . Point somewhere else with `CODITY_DATABASE_URL=postgresql+asyncpg:///.../other_db` .

2. Two fixtures, because concurrency tests need real sessions

Fixture	Mechanism	Used by
<code>db</code>	A connection-bound transaction, rolled back after each test	~90% of tests. Fast and fully isolated.
<code>db_committed</code>	Real commits, truncation teardown	Every test that spawns concurrent sessions

The second fixture is not a convenience. **SKIP LOCKED semantics do not exist inside a single transaction** — a transaction cannot skip its own locks, and two "workers" sharing one session are one worker. Any concurrency test written against the `rollback` fixture passes for the wrong reason and tests nothing. Tests requiring it are marked `@pytest.mark.concurrency` and can be run alone with `make test-concurrency` .

3. Critical tests

These are the evidence for the reliability design. **They are not cut under time pressure.**

Claiming and concurrency

Test	Arrange → Act → Assert
<code>test_concurrent_claims_are_disjoint</code>	100 queued jobs, 8 concurrent sessions → all claim → union == 100, pairwise intersection empty
<code>test_claim_respects_max_concurrency</code>	cap 3, 12 concurrent claimers, 50 jobs → claim → in-flight never exceeds 3 at any sample
<code>test_pause_stops_claiming_resume_restarts</code>	pause mid-drain → claims return 0; resume → claims resume
<code>test_claim_uses_partial_index</code>	EXPLAIN the real claim → references <code>ix_jobs_claim</code> , no Seq Scan, no Sort

Leases, fencing, recovery

Test	Arrange → Act → Assert
<code>test_expired_lease_reclaimed_once</code>	claim, expire lease, two concurrent reapers → exactly one requeue, <code>lease_epoch +1</code>
<code>test_stolen_lease_write_is_fenced</code>	claim at epoch e, reap, re-claim → complete with epoch e → rowcount 0, job untouched
<code>test_zombie_after_same_worker_reclaim_rejected</code>	W1 claims, is reaped, W1 re-claims (epoch e+2), zombie completes with e → rejected — the ABA case
<code>test_orphan_execution_does_not_block_next_attempt</code>	claim, lose the worker, reap, claim again → the second execution insert succeeds

`test_orphan_execution_does_not_block_next_attempt` guards the `closed` CTE in `reap_leases.sql`. `ux_job_executions_open_one` permits one open execution per job; if an abrupt worker death leaves an orphaned open row and the reaper does not close it, the *next* claim's execution insert raises 23505 — and so does every claim after that. The job becomes permanently unexecutable, and it is precisely the flagship crash-recovery demo that wedges it.

Retries and the DLQ

Test	Arrange → Act → Assert
<code>test_backoff_sequence_per_strategy</code>	attempts 1..5 per strategy → <code>run_at</code> deltas within jitter bounds and capped at <code>backoff_max_ms</code>
<code>test_retry_goes_to_scheduled_not_queued</code>	fail with budget remaining → status <code>scheduled</code> , not claimable until <code>run_at</code>
<code>test_exhausted_job_dead_letters_once</code>	<code>max_attempts=2</code> , always-fail handler → exactly one <code>dead_letter_entries</code> row, <code>finished_at</code> set
<code>test_dlq_replay_of_keyed_job_succeeds</code>	dead-letter a job carrying an <code>Idempotency-Key</code> , replay → new job, no unique violation

Backoff is **full jitter**, so these assert bounds, never equality. A test that asserts an exact delay against a jittered schedule is a flaky test that will be deleted by whoever inherits it.

Shutdown

Test	Arrange → Act → Assert
<code>test_sigterm_releases_unstarted_claims</code>	SIGTERM with buffered claims → jobs return to <code>queued</code> , <code>attempt</code> unchanged, nothing lost
<code>test_start_rowcount_zero_cancels_task</code>	release the job between claim and start → the handler is never invoked

The second is the deploy-time double-execution bug: the release commits while the start is in flight, another worker claims the now- `queued` job, and the original handler runs to completion anyway. No lease expiry and no partition required — just a rolling restart.

Scheduling

Test	Arrange → Act → Assert
<code>test_cron_two_schedulers_one_job_per_occurrence</code>	2 schedulers, one due schedule → exactly one job for that <code>scheduled_for</code>
<code>test_cron_advances_when_insert_suppressed</code>	pre-insert the occurrence, tick → <code>next_occurrence_at</code> still advances

The second guards a schedule that dies silently. If `next_occurrence_at` is advanced only when the insert returns a row, a suppressed insert (a prior tick that committed the job then crashed, or an operator's "run now") freezes the cursor: the schedule re-selects, re-conflicts, and does nothing — forever, once per second, with no error anywhere.

API and tenancy

Test	Arrange → Act → Assert
<code>test_idempotency_replay_returns_original</code>	same <code>Idempotency-Key</code> twice → one job row, same id, <code>201</code> both times
<code>test_cross_org_returns_404</code>	a user in org A requests org B's job → 404 , not 403
<code>test_migrations_upgrade_then_downgrade</code>	empty DB → <code>upgrade head</code> → <code>downgrade base</code> → clean
<code>test_state_machine_matches_enum</code>	the DB trigger's edge set == <code>LEGAL_TRANSITIONS</code> in <code>app/domain/enums.py</code>
<code>test_layering_rules</code>	AST-walk every module under <code>app/ : domain/</code> imports no <code>fastapi / app.db / app.api ; services/</code> and <code>repositories/</code> import no <code>fastapi</code>

`test_layering_rules` is what makes the layering rule in `ARCHITECTURE.md` §3 real. An architecture rule nobody checks is a comment, and the specific thing it protects is the worker's ability to import `services/claim.py` without pulling in the web framework.

4. Integration tier

Roughly 20 tests: one 422-shape assertion and one 404 assertion per router, plus the full idempotency and cross-org matrices. They exist to catch envelope drift — a router that returns FastAPI's bare `{"detail": ...}` instead of the error envelope, or `state` where the contract says `status`.

5. No coverage gate

`make cov` prints a coverage report. There is deliberately **no** `--cov-fail-under`.

Coverage percentage earns zero marks and does not measure whether the reliability invariants hold. A gate added late in a build creates pressure to write filler serializer tests at exactly the moment the concurrency tests are at risk — optimising the number instead of the property. The tests in §3 are the ones that matter, and they are named individually so their absence is visible.

6. Writing a new concurrency test

1. Use `db_committed`, and mark it `@pytest.mark.concurrency`.
2. Open **separate sessions** — one per simulated worker. Two coroutines sharing a session are one worker with extra steps.
3. Drive them with `asyncio.gather` so the statements genuinely interleave.
4. Assert a **property** (disjointness, a cap, a rowcount of zero), never a timing.
5. Let the fixture handle teardown; a test that writes outside the fixture must clean up after itself, or it will poison the next run.

Canonical Names

This file is the single source of truth for every name that appears in more than one place: the database, `app/domain/enums.py`, the OpenAPI schema, and the React client. It was written **before** the code it governs.

Every name drift that has to be reconciled late in a project like this — `status` vs `state`, `dead` vs `dead_letter`, `run_at` vs `scheduled_at` vs `next_run_at`, `is_paused` vs `paused`, which direction priority sorts, whether an id is a `uuid` or a `bigint` — is a symptom of not having this file. Nothing here is negotiable in a pull request; changing a name here is a schema change.

`backend/app/domain/enums.py` is the executable copy. **Import from it. Never redefine a literal.**

1. `job_status` — native Postgres `ENUM`

The lifecycle state of a job row. Eight values, no others.

Value	Meaning
<code>scheduled</code>	Not yet eligible. Waiting for <code>run_at</code> . Covers delayed jobs, cron occurrences, and backoff retries .
<code>queued</code>	Eligible now . This is the only status the claim query looks at.
<code>claimed</code>	A worker holds a lease but has not started the handler. <code>attempt</code> not yet incremented.
<code>running</code>	Handler is executing. <code>attempt</code> has been incremented.
<code>completed</code>	Terminal. Handler returned.
<code>failed</code>	Terminal. Non-retryable failure that is not routed to the DLQ.
<code>dead_letter</code>	Terminal. Attempt budget exhausted, or a <code>PermanentError</code> . A <code>dead_letter_entries</code> row exists.
<code>cancelled</code>	Terminal. Cancelled before or during execution.

Terminal set: `completed`, `failed`, `dead_letter`, `cancelled`. Terminal statuses have **zero out-edges** — see `docs/DATABASE.md`. Terminal statuses **require** `finished_at`; the `ck_jobs_terminal_finished` CHECK aborts the transaction if it is omitted.

Not used anywhere: `pending`, `waiting`, `active`, `success`, `error`, `dead`, `retrying`.

2. `execution_status` — native Postgres `ENUM`

The outcome of one *attempt*, recorded on `job_executions`. Seven values.

Value	Meaning
claimed	Row opened at claim time. Open (finished_at IS NULL).
running	Handler started. Open.
succeeded	Handler returned. Closed.
failed	Handler raised. Closed.
timed_out	Exceeded jobs.timeout_ms . Closed. Counts as a failed attempt.
cancelled	Cancelled mid-flight. Closed.
lost	The reaper closed an orphan whose worker never came back. Closed.

succeeded , not completed — the job says completed , the attempt says succeeded , and the difference is load-bearing: a job can complete after several attempts that did not succeed.

3. Timestamps — run_at is the only eligibility clock

Column	Table	Meaning
run_at	jobs	The single eligibility timestamp. The promoter moves scheduled → queued when run_at ≤ now() . Backoff writes a future run_at .
scheduled_for	jobs	Cron occurrence instant only. The nominal time the recurrence rule fired. NULL unless schedule_id is set (ck_jobs_occurrence enforces the biconditional). Never used for eligibility.
next_occurrence_at	job_schedules	The next instant the dispatcher should materialise. Advanced from the nominal scheduled_for , never from now() .
claimed_at / started_at / finished_at	jobs , job_executions	Lifecycle instants.
lease_expires_at	jobs	When the reaper may reclaim.
last_heartbeat_at	workers	Denormalised liveness; the hot check.

Not used anywhere: scheduled_at , next_run_at , execute_at , eta , available_at .

All timestamps are timestamptz . There are no naive timestamps in the schema, in Python, or on the wire.

All time originates from now() inside Postgres — no process compares its own clock to a lease, so clock skew between nodes cannot cause a false reclaim.

4. Queue pause — is_paused and paused_at , always together

```
CONSTRAINT ck_queues_pause CHECK (is_paused = (paused_at IS NOT NULL))
```

Pause: `is_paused = true, paused_at = now()`. Resume: `is_paused = false, paused_at = NULL`. Writing one without the other is rejected by the CHECK. There is no `paused`, `enabled`, or `active` boolean.

Pause blocks **admission only**: in-flight work finishes, and the cron dispatcher does not materialise occurrences into a paused queue.

5. Priority — `smallint`, `-100..100`, HIGHER RUNS FIRST

`ORDER BY priority DESC, run_at ASC, id ASC`. Default `0`.

Two distinct columns, and they are not interchangeable:

Column	Meaning
<code>queues.priority</code>	Inter-queue weight. How often a worker polls this queue relative to its others.
<code>queues.default_priority</code>	The default <code>jobs.priority</code> for jobs <i>created on</i> this queue.
<code>jobs.priority</code>	Intra-queue ordering within the claim.

The direction is stated in the API field description and in the OpenAPI schema, because "priority 1" meaning "most important" is the other half of the industry and a silent disagreement here inverts the whole queue.

6. Fencing — `lease_epoch`

`jobs.lease_epoch bigint NOT NULL DEFAULT 0`. Incremented on **every** ownership change: claim, reclaim, retry, graceful release. Every write a worker makes carries the epoch it claimed under; zero rows updated means the lease was stolen and the result is discarded.

- `lease_epoch` is **the fence**. Nothing else is.
- `lock_version` is trigger-owned, bumped on every update including heartbeats. It can serve ORM optimistic locking; it can **never** be the fence.
- `lease_expires_at > now()` is **not** a fence — it has an ABA hole (see `docs/DESIGN_DECISIONS.md`, ADR-004).

7. `attempt` — **monotonic, incremented at** `claimed` → `running`

`jobs.attempt` starts at `0` and is incremented by exactly one in the guarded `start_job` statement. It is **never** decremented; a `BEFORE UPDATE` trigger rejects `NEW.attempt < OLD.attempt`.

A job released from `claimed` provably never executed, so nothing needs decrementing. This is what makes graceful shutdown free.

`job_executions.attempt_number` is the attempt this row represents — written at claim time as `jobs.attempt + 1`, i.e. the attempt this claim is *about to* become.

8. Retry destination — `scheduled`, not `queued`

A failed attempt with budget remaining goes to `status = 'scheduled'` with a future `run_at`. The claim predicate is `status = 'queued'` **only** — it has no `run_at` term, which is what keeps `ix_jobs_claim` small. A queued job is by definition due.

9. Roles

Two roles: `owner`, `member`. Stored on `organization_members.role`. The `owner/admin/operator/viewer` ladder is designed in `docs/DESIGN_DECISIONS.md` (ADR-014) and deliberately not built.

10. Identifier type, per table

UUIDv7, application-generated. Time-ordered so inserts stay on the rightmost index page, non-enumerable, and generable by the client before the round trip:

`organizations`, `users`, `projects`, `queues`, `retry_policies`, `jobs`, `job_schedules`, `job_batches`, `workers`, `dead_letter_entries`.

bigint GENERATED ALWAYS AS IDENTITY — append-only, high-volume, never referenced externally, and 8 bytes beats 16:

`job_executions`, `job_logs`, `worker_heartbeats`, `queue_stats_minute`.

Natural / composite keys:

Table	Key
<code>organization_members</code>	<code>(organization_id, user_id)</code>
<code>worker_queue_assignments</code>	<code>(worker_id, queue_id)</code>
<code>refresh_tokens</code>	<code>jti</code> (uuid)
<code>idempotency_keys</code>	<code>(organization_id, key)</code>
<code>system_state</code>	<code>name</code> (text: <code>promoter</code> , <code>cron</code> , <code>reaper</code> , <code>retention</code>)

11. Wire vocabulary

Concept	On the wire
Job lifecycle field	<code>status</code> (never <code>state</code>)
List payload	<code>{ "data": [...], "page": {...}, "meta": {...} }</code> (never <code>items</code> , never <code>results</code>)
Page cursor	<code>page.next_cursor</code> , opaque base64 of <code>(created_at, id)</code>
Error	<code>{ "error": { "code", "message", "details", "request_id" } }</code>
Correlation	<code>X-Request-ID</code> header ↔ <code>meta.request_id</code> ↔ <code>jobs.correlation_id</code>
Idempotency	<code>Idempotency-Key</code> request header

Filter query parameter names match column names exactly: `status`, `kind`, `queue_id`, `handler`, `created_after`, `created_before`, `run_after`, `run_before`. Unknown query parameters are a `400 unknown_query_param`, so a typo'd filter fails loudly instead of silently returning everything.

Deliverables Checklist

Required deliverable	Where	Status
Source code with setup instructions	<code>github.com/Surianandhan/Codity</code> , <code>README.md</code>	Done, quickstart verified live
Architecture diagram	<code>docs/ARCHITECTURE.md</code> (Mermaid, rendered below)	Done
ER diagram	<code>docs/DATABASE.md</code> (Mermaid, rendered below)	Done
API documentation	<code>docs/API.md</code> , 30 live endpoints	Done
Design decisions document	<code>docs/DESIGN_DECISIONS.md</code> , 17 ADRs	Done
Automated tests for critical functionality	<code>backend/tests/</code> , 25/25 passing incl. 11 concurrency tests	Done

Repository

Full source, migration history, and commit-by-commit rationale (including the two concurrency bugs found and fixed live) are at:

<https://github.com/Surianandhan/Codity>

Branch `main` , 5 commits, gate green at every commit (ruff, mypy --strict, alembic check, pytest).